

26-Spring深入剖析

今日授课目标:

1. 目标: IoC容器如何实现DI的
2. 目标: Spring的AOP的原理
3. 目标: 搭建Spring源码环境
4. 目标: IoC容器初始化过程中到底都做了哪些事
5. 目标: IoC容器初始化是如何实例化Bean的【循环依赖, 三级缓存】
6. 目标: Spring如何解决循环依赖问题
7. 目标: Spring中用到了那些设计模式

1. Spring核心内容

1.1 什么是Spring?

Spring focuses on the "plumbing" of enterprise applications so that teams can **focus on** application-level business logic, without unnecessary ties to specific deployment environments.



Spring Framework 5

Spring是一个开源框架, **使得企业软件开发变得更加简单。Spring是一个轻量级控制反转(IoC)和面向切面(AOP)的对象容器框架。**

本质: 可以理解为项目中对象的管家

- 设计目的: 解决企业应用开发的复杂性
- 功能核心: 一个控制反转(IoC)和面向切面(AOP)的对象容器。
- 使用范围: 任何Java应用均可使用Spring

优势:

- **简化企业级应用开发:** 使企业级应用开发变得更简单: 主要是解耦!
- **简化类间关系:** 我们面向接口编程, 使用接口而不是使用类, 大大简化了类与类之间的关系(解耦)
- **简化传统配置:** 提供了一种企业应用程序配置的最好方法
- **简化事务:** 超级简单的事务支持@Transactional
- **简化测试:** 使得应用程序更容易测试
- **简化其他框架:** 不仅让自己简化, 还能简化其他框架, 这就是其强大的整合能力, 非常的方便集成各种优秀框架; 这是其设计理念之一, **不与参与任何框架竞争! 做框架与框架之间的胶水**

主要模块:

1. 持久层数据访问: Jdbc, 对象关系映射ORM, 事务, OXM, JMS...
2. Web层: SpringMVC, Servlet, Portlet, WebSocket..

3. 核心容器：Beans, Core, Context, SpEL
4. 测试
5. 其他

1.2 Spring案例

目标：使用Spring来创建Account对象

步骤：

1. 导入依赖坐标
2. 定义Account实体类
3. 编写beans.xml配置文件，配置配置文件，参考官方文档
 - 通过配置创建对象，同样使用的是构造方法创建
 - 设置属性值
4. 编写测试类
 1. 加载xml配置文件，创建Spring的ioc容器
 2. 获取容器中的accountService对象

实现：

1. 创建Maven工程，导入依赖坐标

```
1 <dependencies>
2     <!--spring 的ioc容器jar包-->
3     <dependency>
4         <groupId>org.springframework</groupId>
5         <artifactId>spring-context</artifactId>
6         <version>5.0.7.RELEASE</version>
7     </dependency>
8     <!--单元测试-->
9     <dependency>
10        <groupId>junit</groupId>
11        <artifactId>junit</artifactId>
12        <version>4.12</version>
13    </dependency>
14 </dependencies>
```

2. 创建Account实体类、及三层架构
3. 创建beans.xml配置文件，配置配置文件，参考官方文档

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xsi:schemaLocation="http://www.springframework.org/schema/beans
5       http://www.springframework.org/schema/beans/spring-beans.xsd">
6
7     <!-- 创建对象：id属性：设置对象在容器中的唯一标识、class属性：设置对象的全限定
8     名称，用于反射创建对象 -->
9     <bean id="account" class="com.hero.pojo.Account">
10         <!-- 为创建的对象赋值，前提条件对象中必须有该变量setter方法
```

```

10         name属性: 设置成员变量的名称
11         value属性: 设置成员变量赋的值 -->
12         <property name="id" value="1"/>
13         <property name="name" value="Musk"/>
14         <property name="money" value="50000000"/>
15     </bean>
16     <bean id="accountDao" class="com.hero.dao.AccountDaoImpl"/>
17     <bean id="accountService"
18         class="com.hero.service.AccountServiceImpl">
19         <property name="accountDao" ref="accountDao"/>
20     </bean>
21 </beans>

```

4. 编写测试类: 加载xml配置文件, 创建Spring的ioc容器, 获取容器中的accountService对象

```

1 public class TestSpring {
2     @Test
3     public void test(){
4         //1.创建Spring的ioc容器, ApplicationContext, 通过xml配置文件初始化
5         ApplicationContext ac = new
6         ClassPathXmlApplicationContext("applicationContext.xml");
7         //2.配置XML配置文件applicationContext.xml
8         //3.从ioc容器中取出容器创建好的对象(bean)
9         AccountService accountService =
10        ac.getBean(AccountService.class);
11        Account account = ac.getBean(Account.class);
12        //4.调用执行对象中的方法测试
13        accountService.save(account);
14    }
15 }

```

1.3 基于XML的IoC容器配置

什么是IoC?

IoC: inverse of control (反转控制/控制反转), 我们之前在一个java类中通过new的方式去引入外部资源(其他类等), 这叫正向; 现在Spring框架帮我们去new, 你什么时候需要你去问Spring框架要, 这就是反转(创建对象权利的一个反转), 我们丧失了一个权利(主动创建对象的权利), 但是我们拥有了一个福利(我们不用考虑对象的创建、销毁等问题)。

控制反转 == 对象控制权交给Spring的对象容器

IoC的主要作用: 解决程序耦合, 及管理企业应用中的对象

Bean标签

Bean作用域scope: 对象的使用范围:

- singleton: 默认值, 单例, 项目全局内存中只有一个对象, 一般在使用Spring的时候保持默认值即可
- prototype: 原型的意思, 多例, 每引用或者getBean一次, Spring框架就返回一个新的对象

```
1 <!-- 配置业务层对象: id属性: 设置对象的唯一标识、class属性: 设置类的全限定名称 -->
2 <bean id="accountService" class="com.hero.service.AccountServiceImpl"
  scope="singleton"/>
```

Bean生命周期方法:

- init-method指定对象初始化时执行的方法
- destroy-method指定对象销毁时执行的方法

```
1 <bean id="accountService" class="com.hero.service.AccountServiceImpl"
  scope="singleton"
2   init-method="init" destroy-method="destroy"/>
```

注入Bean的三种配置方式:

- 第一种: 框架使用反射创建对象, 通过配置的class全限定名

```
1 <bean id="accountDao" class="com.hero.dao.AccountDaoImpl"/>
```

- 第二种: 使用静态工厂方法创建对象

```
1 <!-- factory-method属性: 设置工厂中创建对象的方法 -->
2 <bean id="accountDaoTwo" class="com.hero.factory.StaticFactory" factory-
  method="createAccountDao"/>
```

- 第三种: 使用实例工厂方法创建对象

```
1 <!-- 首先将实例工厂的对象注入到Spring的容器, 接着使用实例工厂来创建对象
2   factory-bean属性: 设置实例工厂的bean对象唯一标识
3   factory-method属性: 设置工厂中创建对象的方法-->
4 <!--实例工厂-->
5 <bean id="instanceFactory" class="com.hero.factory.InstanceFactory"/>
6 <!--实例工厂创建的对象-->
7 <bean id="accountThree" factory-bean="instanceFactory" factory-
  method="createAccountDao"/>
```

什么是依赖注入DI?

ID依赖注入: Dependency Injection, 它是 Spring 核心 IoC 的体现之一

我们的应用程序 通过IoC 控制反转, 把对象的创建交给了 Spring对象容器, 但是代码中不可能没有依赖。IoC解耦只是降低他们的依赖关系, 但不会消除。依赖的管理需要以DI来处理。

那么如何管理依赖?

一共有三种方式:

- 第一种: 构造函数注入

```

1 <bean id="accountService" class="com.hero.service.AccountServiceImpl">
2     <!-- 构造方法注入参数
3     name属性: 设置AccountServiceImpl构造方法中那个参数被注入
4     ref属性: 设置注入的引用对象accountDao
5     value属性: 设置值 -->
6     <constructor-arg name="accountDao" ref="accountDao"/>
7     <constructor-arg name="id" value="11"/>
8     <constructor-arg name="name" value="古力娜扎"/>
9 </bean>

```

- 第二种: set方法注入

```

1 <bean id="accountService" class="com.hero.service.AccountServiceImpl">
2     <!-- set方法注入参数
3     name属性: 设置AccountServiceImpl 的set方法中那个成员变量被注入
4     ref属性: 设置注入的引用对象accountDao
5     value属性: 设置值 -->
6     <property name="accountDao" ref="accountDao"/>
7     <property name="id" value="11"/>
8     <property name="name" value="迪丽热巴"/>
9     <!--注入数组-->
10    <property name="myStrs">
11        <array>
12            <value>str1</value>
13            <value>str2</value>
14            <value>str3</value>
15        </array>
16    </property>
17    <!--注入list集合-->
18    <property name="myList">
19        <list>
20            <value>list1</value>
21            <value>list2</value>
22            <value>list3</value>
23        </list>
24    </property>
25    <!--注入set集合-->
26    <property name="mySet">
27        <set>
28            <value>set1</value>
29            <value>set2</value>
30            <value>set3</value>
31        </set>
32    </property>
33    <!--注入map-->
34    <property name="myMap">
35        <map>
36            <entry key="key1" value="v1"/>
37            <entry key="key2" value="v2"/>
38            <entry key="key3" value="v3"/>
39        </map>
40    </property>
41 </bean>

```

- 第三种：使用名称空间p注入，本质上与set方法注入的原理一致，都是通过set方法注入

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:p="http://www.springframework.org/schema/p"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5       xsi:schemaLocation="http://www.springframework.org/schema/beans
6       http://www.springframework.org/schema/beans/spring-beans.xsd">
7     <!-- 使用P名称空间的p标签属性注入 -->
8     <bean id="accountService"
9           class="com.hero.service.AccountServiceImpl"
10          p:id="11" p:name="迪丽热巴" p:accountDao-ref="accountDao"/>
11 </beans>
```

1.4 基于注解的IoC容器配置

在企业真实开发中，配置文件中Bean标签会非常多，非常难以维护。怎么办？

- 基于注解的 IoC 配置，需要有这样一个认知：即 注解 和 xml 配置实现的功能一样，都是要降低程序间耦合。只是配置形式不一样。
- 注解和XML配置内容可以互相置换。实际开发中，到底用 XML 还是注解 各有优劣 没有那种是最好的。

使用注解的前提：配置注解扫描

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xmlns:context="http://www.springframework.org/schema/context"
5       xsi:schemaLocation="http://www.springframework.org/schema/beans
6       http://www.springframework.org/schema/beans/spring-beans.xsd
7       http://www.springframework.org/schema/context
8       http://www.springframework.org/schema/context/spring-context.xsd">
9     <!-- 配置包内的注解扫描，base-package 属性指定扫描包地址 -->
10
11     <context:component-scan base-package="com.hero"/>
12 </beans>
```

@Component(value="")

1. 注解作用：把AccountDao实现类对象交由Spring IOC容器管理

- 相当于XML配置文件中的Bean标签

```
1 <bean id="accountDao" class="com.hero.dao.AccountDao"/>
```

2. 标记在类上，也可以放在接口上
3. 注解Value属性：相当于bean标签id，对象在IOC容器中的唯一标识，可以不写，默认值是当前类首字母缩写的类名。
4. **三个衍生注解分别是：@Controller，@Service，@Repository**
 - 作用与@Component注解一样
 - 设计初衷增加代码的可读性，体现在三层架构上
 - @Controller一般标注在表现层
 - @Service一般标注在业务层
 - @Repository一般 标注在持久层

@Autowired

1. 注解作用：自动将ioc容器中的对象注入到当前的成员变量中。按照变量的**数据类型**注入
 - 默认按照变量数据类型注入，如果数据类型是接口，注入接口实现类。
 - 相当于XML配置文件中的property标签

```
1 | <property name="accountDao" ref="accountDao"/>
```

2. 标记在成员变量或set方法上
3. 注意事项：
 - 它只能注入其他的bean类型
 - 成员变量的接口数据类型，有多个实现类的时候，要使用bean的id注入，否则会报错。
 - Spring框架提供的注解
4. 指定Bean的id，使用@Qualifier的注解配合，@Qualifier注解的value属性指定bean的id

@Resource(name="")

1. 注解作用：相当于@Autowired的注解与@Qualifier的注解合并了。直接按照bean的id注入。
 - 相当于XML配置文件中的property标签

```
1 | <property name="accountDao" ref="accountDao"/>
```

2. name属性：指定bean的id
3. 如果不写name属性则按照变量数据类型注入，数据类型是接口的时候注入其实现类。如果定义name属性，则按照bean的id注入。
4. 标记在成员变量或set方法上
5. Java的JDK提供本注解

@Configuration

1. 注解作用：作用等同于beans.xml配置文件，当前注解声明的类中，编写配置信息，所以我们将@Configuration声明的类称之为配置类
2. 标记在类上

@Import

- 注解作用：导入其他配置类(对象)
- 标记在类上
- 相当于XML配置文件中的标签

```
1 <import resource="classpath:applicationContext-dao.xml"/>
```

@PropertySource

- 注解作用：引入外部属性文件(db.properties)
- 相当于XML配置文件中的context:property-placeholder标签
- 标记在类上

```
1 <context:property-placeholder location="classpath:jdbc.properties"/>
```

@Value

- 注解作用：给简单类型变量赋值
- 相当于XML配置文件中的标签
- 标记在成员变量或set方法上

```
1 <property name="driverClass" value="${jdbc.driver}"/>
```

@Bean

- 注解作用：将方法的返回值存储到Spring IOC容器中
- 相当于XML配置文件中的标签
- 标记在配置类中的方法上

```
1 <bean id="accountDao" class="com.hero.dao.AccountDaoImpl"/>
```

@ComponentScan("com.hero")

- 注解作用：扫描注解，相当于beans.xml中的注解扫描配置 `<context:component-scan base-package="com.hero"/>`
- 相当于XML配置文件中的context:component-scan标签
- 标记在配置类上

```
1 <context:component-scan base-package="com.hero"/>
```

其他注解

@Scope("prototype")

- 注解作用：指定对象的作用范围：单例模式(singleton)还是多例模式(prototype)
- 标记在类上，配合@Component使用

生命周期注解

- @PostConstruct ==> init-method
- @PreDestroy ==> destroy-method

1.5 Spring中的AOP

什么是动态代理？

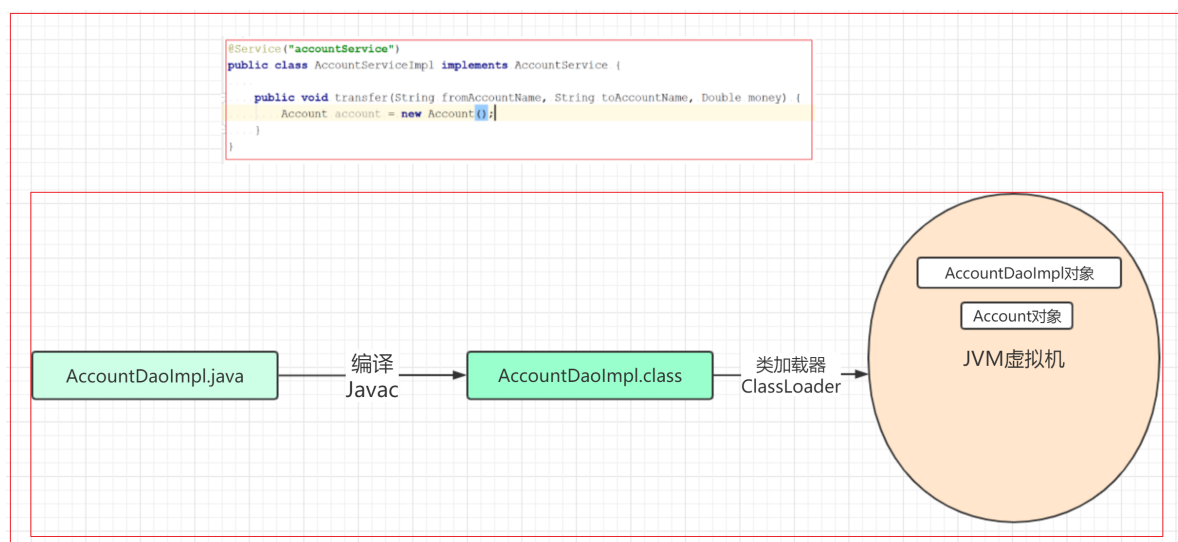
动态代理就是，在程序运行期，创建目标对象的代理对象，并对目标对象中的方法进行功能性增强的一种技术。

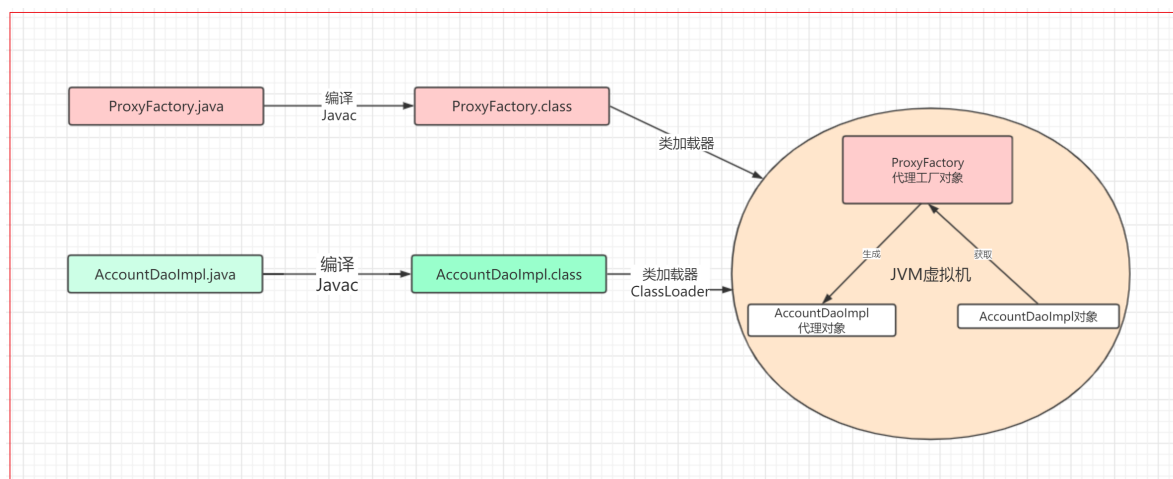
在生成代理对象的过程中，目标对象不变，代理对象中的方法是目标对象方法的增强方法。可以理解为运行期间，对象中方法的动态拦截，在拦截方法的前后执行功能操作。

创建代理对象的两个方法：

- 方法01-基于接口的动态代理
 - 提供者：JDK
 - 使用JDK官方的Proxy类创建代理对象
 - 注意：代理的目标对象必须实现接口
- 方法02-基于类的动态代理
 - 提供者：第三方 CGLib
 - 使用CGLib的Enhancer类创建代理对象
 - 注意：如果报 asmxxxx 异常，需要导入 asm.jar包

```
1 //JDK动态代理
2 Proxy.newProxyInstance(三个参数);
3 //CGLib动态代理
4 Enhancer.create(两个参数);
```





什么是AOP?

AOP (Aspect Oriented Programming) **面向切面编程**, 是通过**运行期动态代理**, 实现程序功能统一维护的技术。

AOP是OOP的延续, 也是Spring框架中的一个重要内容。利用AOP可以对业务逻辑的各个部分进行隔离, 从而使得业务逻辑各部分之间的耦合度降低, 提高程序的可重用性, 同时提高了程序开发的效率。

简单来说: 它就是把我们程序重复的代码抽取出来, 在需要的时候, 利用动态代理, 在不修改源码的情况下, 对我们的已有方法进行功能的增强。这就相当于方法的拦截器。批量方法功能的动态增强。

作用: 在程序运行期间, 不修改源码, 对已有方法进行增强。

优势: 减少重复代码, 提高开发效率, 方便维护

AOP经典案例: 记录日志、性能监控、**事务管理**、权限控制、编写插件...

基于XML配置的AOP【声明式】

前面介绍了AOP, 面向切面编程是一种思想, 在Spring中AOP更加强大, 无需编写复杂代码, 通过XML配置即可实现面向切面编程的效果。

AOP相关概念

想要掌握Spring的AOP, 首先的理解AOP中的一些基础概念:

- **Pointcut(切入点):** 项目中有很多类, 每个类有很多方法, 这么多方法, 如何定位到需要增强功能的方法呢? 靠的是切入点
- **Aspect(切面):** 切面由切点和增强组成, **切面 = 切点 + 增强**, 也可以= 切点 + 方位信息 + 横切逻辑, 还可以= 连接点 + 横切逻辑
- **JoinPoint(连接点):** 横切程序执行的特定位置, 比如: 类中某个方法调用前、调用后, 方法抛出异常后等
- **Target(目标对象):** 增强逻辑的目标类
- **Weaving(织入):** 织入是将增强逻辑/横切逻辑添加到目标类具体连接点上的过程
- **Proxy(代理):** 一个类被AOP织入增强后, 就产出了一个代理类, 它是融合了原类和增强逻辑的代理类。

- **Advice(通知/增强)**: 增强的第一层意思就是你的横切逻辑代码（增强逻辑代码）

本质：把横切逻辑增强到连接点（切点和方位信息都是为了确定连接点）上

常用四中通知/增强类型：

前置通知(before): 用于配置前置通知。指定增强的方法在切入点方法之前执行

- 执行时间点：切入点方法执行之前执行
- method:用于指定通知类中的增强方法名称
- ponitcut-ref: 用于指定切入点的表达式的引用
- poinitcut: 用于指定切入点表达式

后置通知(afterReturn): 用于配置后置通知。指定增强的方法在切入点方法之后执行

- 执行时间点：切入点方法正常执行之后。抛出异常就不执行了
- method:用于指定通知类中的增强方法名称
- ponitcut-ref: 用于指定切入点的表达式的引用
- poinitcut: 用于指定切入点表达式

异常通知(afterThrowing): 用于配置异常通知，方法抛出异常执行

- 执行时间点：切入点方法执行产生异常后执行。它和后置通知只能执行一个
- method:用于指定通知类中的增强方法名称
- ponitcut-ref: 用于指定切入点的表达式的引用
- poinitcut: 用于指定切入点表达式

最终通知(after): 用于配置最终通知，在方法执行完毕的最后执行

- 执行时间点：无论切入点方法执行时是否有异常，它都会在其后面执行
- method:用于指定通知类中的增强方法名称
- ponitcut-ref: 用于指定切入点的表达式的引用
- poinitcut: 用于指定切入点表达式

环绕通知(around)

用于配置环绕通知。它是 spring 框架为我们提供的一种可以在代码中手动控制增强代码什么时候执行的方式。

- method:用于指定通知类中的增强方法名称
- ponitcut-ref: 用于指定切入点的表达式的引用
- poinitcut: 用于指定切入点表达式
- 注意：通常情况下，环绕通知都是独立使用的

举个栗子：

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xmlns:aop="http://www.springframework.org/schema/aop"
5       xsi:schemaLocation="http://www.springframework.org/schema/beans
6       http://www.springframework.org/schema/beans/spring-beans.xsd
7       http://www.springframework.org/schema/aop
```

```

8      http://www.springframework.org/schema/aop/spring-aop.xsd">
9
10     <!--配置业务层实现类-->
11     <bean id="accountService" class="com.hero.service.AccountServiceImpl"/>
12     <!--配置日志通知(增强)工具对象，交由Spring ioc容器管理-->
13     <bean id="logger" class="com.hero.utils.LogUtils"/>
14     <!-- 配置AOP -->
15     <aop:config>
16         <!-- 配置切面：id属性：设置当前切面的唯一标识、ref属性：指定当前切面的增强类(日
            志增强工具类) -->
17         <aop:aspect id="loggerAdvice" ref="logger">
18             <!-- 配置切入点 -->
19             <aop:pointcut id="logPointcut"
20                 expression="execution(public *
com.hero.*.AccountServiceImpl.*(..))"/>
21             <!-- 配置前置增强：在方法执行前执行-->
22             <aop:before method="beforePrintLog" pointcut-ref="logPointcut"/>
23             <!-- 配置配置后置通知：在方法执行完毕之后执行(即便抛出异常还是执行) -->
24             <aop:after-returning method="afterReturnPrintLog" pointcut-
ref="logPointcut"/>
25             <!-- 配置异常通知增强：在方法抛出异常后执行 -->
26             <aop:after-throwing method="afterThrowingPrintLog" pointcut-
ref="logPointcut"/>
27             <!-- 配置最终通知增强：在方法执行后执行 -->
28             <aop:after method="afterPrintLog" pointcut-ref="logPointcut"/>
29
30             <!--配置环绕通知：一个顶四个-->
31             <aop:around method="aroundPrintLog" pointcut-ref="logPointcut"/>
32         </aop:aspect>
33     </aop:config>
34 </beans>

```

```

1  /**
2   * 日志根据类，业务逻辑中的横切逻辑(通知，增强，)
3   */
4  public class LogUtils {
5
6      //方法之前输出日志
7      public void beforePrintLog(){
8          System.out.println("在方法之前执行");
9      }
10     //方法之后输出日志
11     public void afterReturnPrintLog(){
12         System.out.println("在方法之后执行，有异常就不执行");
13     }
14     //方法抛出异常执行
15     public void afterThrowingPrintLog(){
16         System.out.println("在方法抛出异常之后执行");
17     }
18     //方法的最后执行，不论有没有异常
19     public void afterPrintLog(){
20         System.out.println("在方法最后执行，不管有没有异常都执行");
21     }
22     //配置环绕通知，相当于自定义通知增强
23     public Object aroundPrintLog(ProceedingJoinPoint joinPoint){

```

```

24     Object obj = null;
25     try {
26         System.out.println("在方法之前执行");
27
28         obj = joinPoint.proceed();//调用方法，这相当于调用被代理类的被增强的方法。有点类似，method.invoke()
29
30         System.out.println("在方法之后执行，有异常就不执行");
31     } catch (Throwable throwable) {
32         throwable.printStackTrace();
33
34         System.out.println("在方法抛出异常之后执行");
35     } finally {
36         System.out.println("在方法最后执行，不管有没有异常都执行");
37     }
38     return obj;
39 }
40 }

```

基于注解配置的AOP

首先必须要在XML配置开启AOP注解支持

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <beans xmlns="http://www.springframework.org/schema/beans"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xmlns:aop="http://www.springframework.org/schema/aop"
5      xmlns:context="http://www.springframework.org/schema/context"
6      xsi:schemaLocation="http://www.springframework.org/schema/beans
7          http://www.springframework.org/schema/beans/spring-beans.xsd
8          http://www.springframework.org/schema/aop
9          http://www.springframework.org/schema/aop/spring-aop.xsd
10         http://www.springframework.org/schema/context
11         http://www.springframework.org/schema/context/spring-context.xsd">
12      <!--配置开启注解扫描-->
13      <context:component-scan base-package="com.hero"/>
14      <!-- 开启aop的注解支持 -->
15      <aop:aspectj-autoproxy/>
16  </beans>

```

比XML配置更简单了，相当于将XML中的AOP配置移入到增加类中

```

1  /**
2   * 日志根据类，业务逻辑中的横切逻辑(通知，增强)
3   * @Aspect 注解作用：标记当前类的为日志增强通知切面类
4   * 作用相当于标签：<aop:aspect id="loggerAdvice" ref="logger">
5   */
6  @Component("logger")
7  @Aspect()
8  public class LogUtils {
9      //注解配置切入点表达式，引用只需要调用当前方法即可
10     @Pointcut("execution(public * com.hero.service.AccountServiceImpl.*(..))")

```

```

11     public void pointcut(){}
12
13     //value属性值：可以是切入点表达式，也可以是切入点表达式的引用
14     //@Before("pointcut()")
15     public void beforePrintLog(){//方法之前输出日志
16         System.out.println("在方法之前执行");
17     }
18
19     //@AfterReturning("pointcut()")
20     public void afterReturnPrintLog(){//方法之后输出日志
21         System.out.println("在方法之后执行，有异常就不执行");
22     }
23
24     //@AfterThrowing("pointcut()")
25     public void afterThrowingPrintLog(){//方法抛出异常执行
26         System.out.println("在方法抛出异常之后执行");
27     }
28
29     //@After("pointcut()")
30     public void afterPrintLog(){//方法的最后执行，不论有没有异常
31         System.out.println("在方法最后执行，不管有没有异常都执行");
32     }
33
34     @Around("pointcut()")//配置环绕通知，相当于自定义通知增强
35     public Object aroundPrintLog(ProceedingJoinPoint joinPoint){
36         Object obj = null;
37         try {
38             System.out.println("在方法之前执行");
39             obj = joinPoint.proceed();//调用方法，这相当于调用被代理类的被增强的方
40             //法。有点类似，method.invoke()
41             System.out.println("在方法之后执行，有异常就不执行");
42         } catch (Throwable throwable) {
43             throwable.printStackTrace();
44
45             System.out.println("在方法抛出异常之后执行");
46         } finally {
47             System.out.println("在方法最后执行，不管有没有异常都执行");
48         }
49         return obj;
50     }
51 }

```

1.6 Spring中的事务

事务基本特性：原子性Atomicity，一致性Consistency，隔离性Isolation，持久性Durability

事务并发问题：脏读、不可重复读、虚读 / 幻读

事务隔离级别：RU读未提交、RC读已提交、RR可重复读、Serializable 串行化

- 隔离级别安全性：serializable > RR > RC > RU
- 隔离级别性能：serializable < RR < RC < RU

常见数据库的默认隔离级别：

- MySql: `RR`
- Oracle: `RC`

事务传播行为：

事务往往加载service层方法上的。业务简单的场景，事务直接绑定在方法service或dao层方法。业务场景复杂，可能会涉及service层多层方法调用。

比如：方法A()直接调用service层方法B()，此时A()和B()都有自己的事务控制，那么，事务应该如何进行控制呢？如果不控制，相互调用的时候就会有问题。因此事务传播行为就出现了。**传播行为负责协商A和B关于事务的处理机制。**

常见的传播行为如下：

- **REQUIRED**：如果当前没有事务，就新建一个事务，如果已经存在一个事务中，加入到这个事务中。一般的选择（默认值）
- **SUPPORTS**：支持当前事务，如果当前没有事务，就以非事务方式执行（没有事务）
- **MANDATORY**：使用当前的事务，如果当前没有事务，就抛出异常
- **REQUIRES_NEW**：新建事务，如果当前在事务中，把当前事务挂起。
- **NOT_SUPPORTED**：以非事务方式执行操作，如果当前存在事务，就把当前事务挂起
- **NEVER**：以非事务方式运行，如果当前存在事务，抛出异常
- **NESTED**：如果当前存在事务，则在嵌套事务内执行。如果当前没有事务，则执行REQUIRED类似的操作。

基于XML的声明式事务控制

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xmlns:context="http://www.springframework.org/schema/context"
5       xmlns:aop="http://www.springframework.org/schema/aop"
6       xmlns:tx="http://www.springframework.org/schema/tx"
7       xsi:schemaLocation="http://www.springframework.org/schema/beans
8       http://www.springframework.org/schema/beans/spring-beans.xsd
9       http://www.springframework.org/schema/context
10      http://www.springframework.org/schema/context/spring-context.xsd
11      http://www.springframework.org/schema/aop
12      http://www.springframework.org/schema/aop/spring-aop.xsd
13      http://www.springframework.org/schema/tx
14      http://www.springframework.org/schema/tx/spring-tx.xsd">
15     <!-- 配置开启组件扫描 -->
16     <context:component-scan base-package="com.hero"/>
17     <!-- 配置JdbcTemplate 对象，交由Spring的ioc容器管理 -->
18     <bean id="jdbcTemplate"
19           class="org.springframework.jdbc.core.JdbcTemplate">
20         <constructor-arg name="dataSource" ref="dataSource"/><!-- 配置数据源 -->
21     </bean>
```

```

17     <context:property-placeholder location="db.properties"/><!-- 引入外部配置
属性文件 -->
18     <!--配置数据源-->
19     <bean id="dataSource"
class="org.springframework.jdbc.datasource.DriverManagerDataSource">
20         <property name="driverClassName" value="{jdbc.driver}"/>
21         <property name="url" value="{jdbc.url}"/>
22         <property name="username" value="{jdbc.username}"/>
23         <property name="password" value="{jdbc.password}"/>
24     </bean>
25     <!--配置事务管理器【对事务的增强】-->
26     <bean id="txManger"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
27         <property name="dataSource" ref="dataSource"/><!--配置数据源-->
28     </bean>
29     <!--配置AOP声明式事务-->
30     <aop:config>
31         <!-- 配置aop的通知增强, advice-ref属性: 设置外部标签的通知增强, pointcut属
性: 配置切入点-->
32         <aop:advisor advice-ref="txAdvice" pointcut="execution( *
com.hero.service.*.*(..))"/>
33     </aop:config>
34     <!-- 配置声明式事务通知 id属性: 为当前通知标记唯一标识, transaction-manager属性:
设置当前事务通知的事务管理器-->
35     <tx:advice id="txAdvice" transaction-manager="txManger">
36         <!-- 配置事务属性 -->
37         <tx:attributes>
38             <!-- 批量设置切入点方法的事务属性
39             name属性: 指定业务层的方法名称, 可以使用*通配符
40             propagation属性: 事务的传播行为
41             read-only属性: 是否只读事务, 默认false
42             isolation属性: 事务的隔离级别, 默认使用数据库的隔离级别
43             timeout属性: 指定超时时间, 默认-1秒, 永不超时
44             rollback-for属性: 用于指定一个异常, 当执行产生该异常时, 事务回滚。没有
默认值, 任何异常都回滚
45             no-rollback-for属性: 用于指定一个异常, 当执行产生该异常时, 事务不回
滚。没有默认值, 任何异常都回滚-->
46             <tx:method name="*" propagation="REQUIRED" read-only="false"/>
47             <tx:method name="find*" propagation="SUPPORTS" read-
only="true"/>
48         </tx:attributes>
49     </tx:advice>
50 </beans>

```

基于注解的事务控制

首先, 确保事务管理器已经配置, 并开启事务注解驱动


```

1 <!-- 配置事务管理器，事务管理的id必须是transactionManager -->
2 <bean id="transactionManager"
  class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
3   <property name="dataSource" ref="dataSource"/><!--配置数据源-->
4 </bean>
5 <!--开启事务注解驱动-->
6 <tx:annotation-driven/>

```

在业务层需要加事务的方法上标注事务注解@Transactional

该注解的属性和 xml 中的属性含义一致。该注解可以出现在接口上，类上和方法上。

- 出现在接口上，表示该接口的所有实现类都有事务支持。
- 出现在类上，表示类中所有方法有事务支持
- 出现在方法上，表示当前方法有事务支持

以上三个位置的优先级：方法 > 类 > 接口

```

* 转账
* @param fromName
* @param toName
* @param money
* 1. 查询出汇款人信息
* 2. 查出收款人信息
* 3. 汇款人减去汇款金额
* 4. 收款人加上汇款金额
* 5. 持久化两个账户的信息
*/
@Transactional(readOnly = false, propagation = Propagation.REQUIRED)
public void transfer(String fromName, String toName, Double money) throws SQLException {
    // 方法执行前
    /** 1. 查询出汇款人信息
    Account fromAccount = accountDao.findByName(fromName);
    /** 2. 查出收款人信息
    Account toAccount = accountDao.findByName(toName);
    /** 3. 汇款人减去汇款金额
    fromAccount.setMoney(fromAccount.getMoney() - money);
    /** 4. 收款人加上汇款金额
    toAccount.setMoney(toAccount.getMoney() + money);
    /** 5. 持久化两个账户的信息
    int result1 = accountDao.update(fromAccount);
    System.out.println(result1);
    // 手动异常
    int i = 1/0;
    int result2 = accountDao.update(toAccount);
    System.out.println(result2);
    // 方法执行后
}

```

2. 案例：手写一个IoC容器框架

目标：自定义IoC对象容器，管理对象，实现三层架构解耦

需求：使用IoC来管理项目中的所有对象，通过配置注入对象到容器（map）对象工厂！

分析需求：

- Spring的容器的本质其实就是一个BeanFactory，对象工厂的一大作用就是解耦，经常被使用，比如SqlSessionFactory
- 容器其实是用Map集合来装载对象
- 在实际开发中，我们可以把三层的对象都使用配置文件配置起来，当启动服务器应用加载的时候，让一个类中的方法通过读取配置文件，把这些对象创建出来并存起来
- 在使用的时候，直接拿过来用就好了，而不是去直接创建
- 这个读取配置文件，创建和获取三层对象的类就是Bean工厂

步骤:

1. 实现框架

1. 导入依赖
2. 创建Bean工厂类BeanFactory
3. 通过工厂类解析配置文件，反射创建对象并存入Map集合

2. 使用框架

1. 编写三层架构代码
2. 定义配置文件
3. 读取配置文件，并初始化IoC容器
4. 从容器中取出对象，并调用对象方法

实现框架：攻城狮

1. 导入依赖坐标

```
1  <dependencies>
2      <!--单元测试Jar包-->
3      <dependency>
4          <groupId>junit</groupId>
5          <artifactId>junit</artifactId>
6          <version>4.12</version>
7          <scope>test</scope>
8      </dependency>
9      <!-- dom4j xml解析jar包 -->
10     <dependency>
11         <groupId>dom4j</groupId>
12         <artifactId>dom4j</artifactId>
13         <version>1.6.1</version>
14     </dependency>
15     <!-- XPATH的jar包 -->
16     <dependency>
17         <groupId>jaxen</groupId>
18         <artifactId>jaxen</artifactId>
19         <version>1.1.6</version>
20     </dependency>
21 </dependencies>
```

2. 创建Bean工厂类：初始化Bean工厂时创建所有的对象，装载到工厂

```
1  public class BeanFactory {
2      //定义一个map集合，用于存放初始化后的对象
3      private static Map<String, Object> map = new HashMap<String, Object>
4      ();
5
6      //初始化容器，静态代码块，在创建对象之前执行
7      static {
8          //1.手动写一个配置文件beans.xml
9          //2.使用类加载器读取配置文件beans.xml输入流
10         InputStream is =
11             BeanFactory.class.getClassLoader().getResourceAsStream("beans.xml");
12         //3.创建xml文件解析器SAXReader对象，用来解析XML文件
13         SAXReader saxReader = new SAXReader();
```

```

12         try {
13             Document document = saxReader.read(is);
14             //4.用XPath取出配置文件中的所有Bean标签，返回元素的集合对象
15             List<Element> elementNodes = document.selectNodes("//bean");
16             //5.循环遍历标签list集合，取出Bean标签的id和class
17             for (Element node : elementNodes) {
18                 String idValue = node.attributeValue("id");
19                 String classValue = node.attributeValue("class");
20                 //6.反射创建对象，装进自定义的spring容器
21                 Class<?> clazz = Class.forName(classValue);
22                 Object classObject = clazz.newInstance();
23                 //将对象存入容器中
24                 map.put(idValue, classObject);
25             }
26         } catch (Exception e) {
27             e.printStackTrace();
28         }
29     }
30 }
31
32 /**
33  * 7.Bean工厂对外提供方法，获取自定义容器中的对象
34  * 根据bean在spring的容器中的唯一标识取出该对象
35  */
36 public static Object getBean(String id){
37     return map.get(id);
38 }
39 }

```

使用框架：码农

1. 创建beans.xml配置文件

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans>
3     <!--配置持久层对象，配置的顺序必须是 1.accountDao => 2.accountService =>
4     3.accountController -->
5     <bean id="accountDao" class="com.hero.dao.AccountDaoImpl"></bean>
6     <bean id="accountService"
7     class="com.hero.service.AccountServiceImpl"></bean>
8     <bean id="accountController"
9     class="com.hero.controller.AccountController"></bean>
10 </beans>

```

2. 使用hero-io-frame核心API接管所有对象

3. 编写测试代码

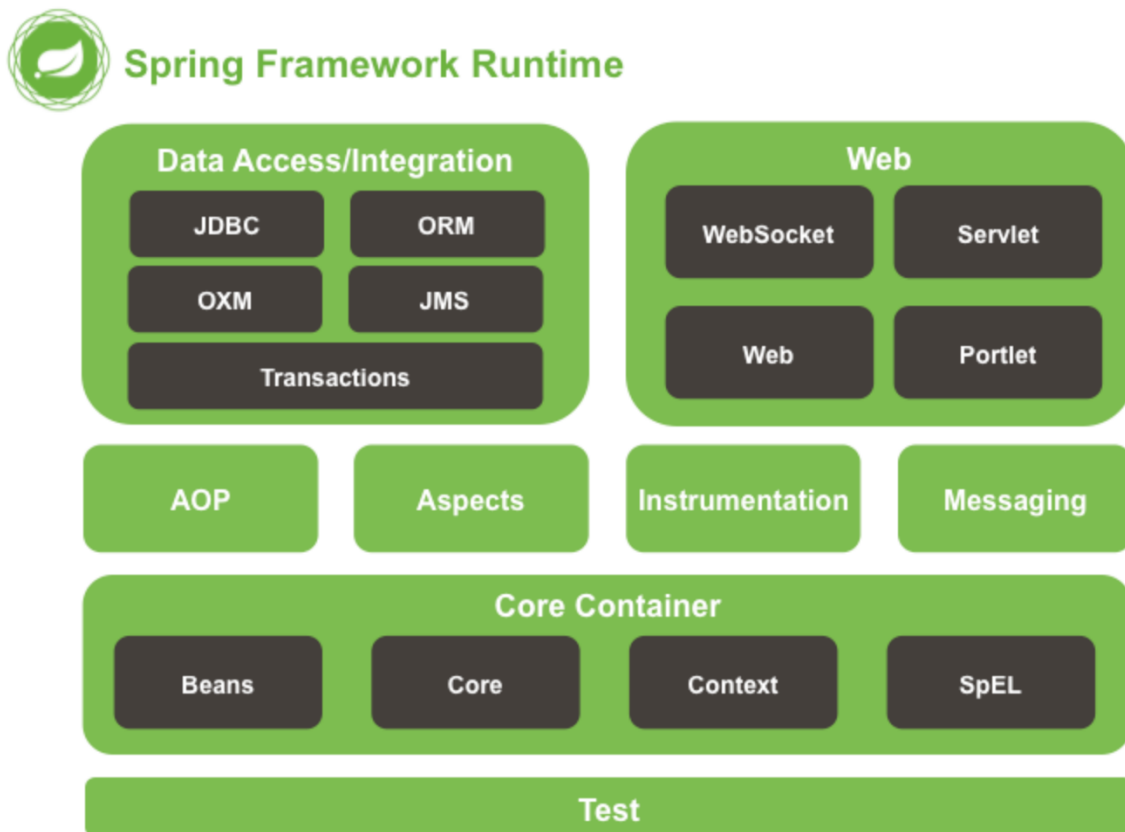
```

1 public class CustomIocTest {
2     @Test
3     public void test(){
4         //通过Beanfactory获取对象
5         AccountController accountController = (AccountController)
        BeanFactory.getBean("accountController");
6         //调用对象中的方法，执行测试
7         accountController.saveAccount(new Account(1,"美国总统01d拜
        登",50000.0));
8     }
9 }

```

3. Spring高级别架构图

3.1 分层模块架构图



数据访问与集成:

- spring-jdbc: 提供JDBC主要实现模块，用于简化JDBC操作
- spring-tx: spring-jdbc事务管理
- spring-orm: 主要集成Hibernate5, JPA
- spring-oxm: 将java对象映射成XML数据，或将XML映射为Java对象
- spring-jms: 发送和接受消息 (MQ)

web模块:

- spring-web: 提供了最基础的Web支持，主要建立在核心容器上

- spring-webmvc: 实现了Spring MVC的Web应用
- spring-websocket: 主要与前端页的全双工通讯协议
- spring-webflux: 一个新的非阻塞式Web框架(5.0中引入)

切面编程:

- spring-aop: 面向切面编程, CGLB,JDKProxy
- spring-aspects: 集成AspectJ, Aop应用框架
- spring-instrument (工具): 动态Class Loading模块
- spring-messaging: 4.0加入的模块, 主要集成基础报文传送应用

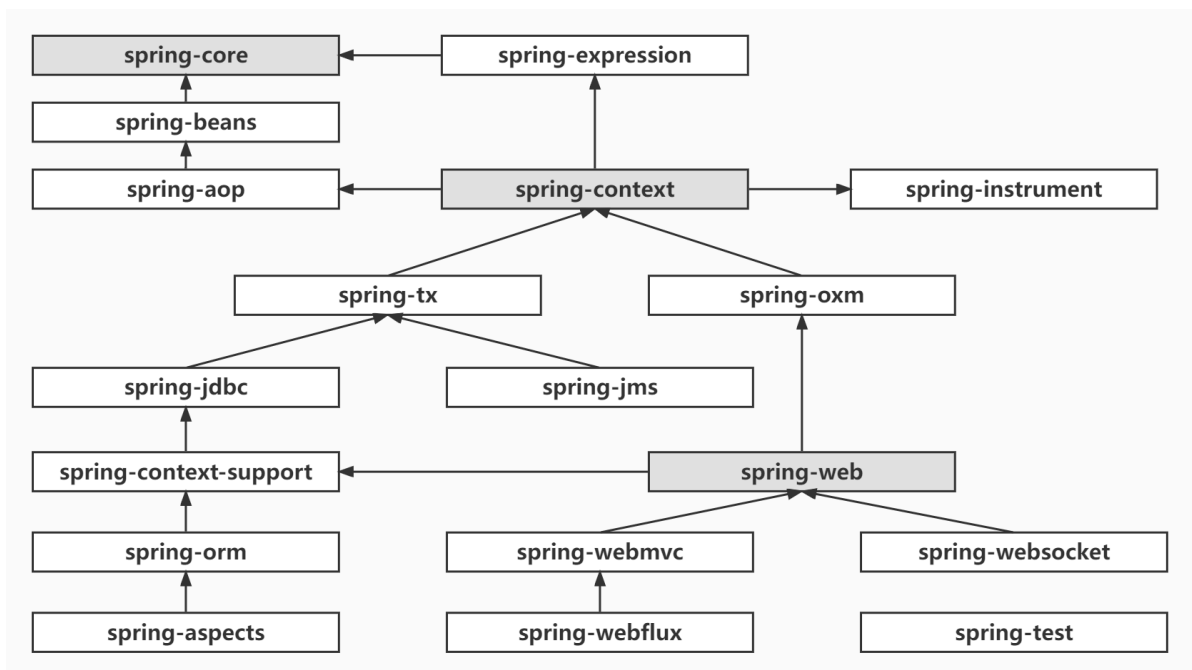
核心模块

- spring-core: 核心模块; 供了框架的基本组成部分, 包括控制反转 (Inversion of Control, IOC) 和依赖注入 (Dependency Injection, DI) 功能
- spring-beans: Bean: 提供了BeanFactory, 是工厂模式的一个经典实现
- spring-context: 上下文, 即IoC容器ApplicationContext
- spring-context-support: 对IoC的扩展以及IoC子容器
- spring-context-indexer: 类管理组件和Classpath扫描
- spring-expression: 表达式语句

测试模块:

- spring-test: 测试组件 (JUnit、Mock)

3.2 模块依赖关系图



对应的源码工程:

- >  spring-aop
- >  spring-aspects
- >  spring-beans
- >  spring-context
- >  spring-context-indexer
- >  spring-context-support
- >  spring-core
- >  spring-expression
- >  spring-framework-core-example
- >  spring-instrument
- >  spring-jcl
- >  spring-jdbc
- >  spring-jms
- >  spring-messaging
- >  spring-orm
- >  spring-oxm
- >  spring-test
- >  spring-tx
- >  spring-web
- >  spring-webflux
- >  spring-webmvc
- >  spring-websocket

4. Spring IoC源码阅读

4.1 源码环境编译与构建

源码调试环境搭建步骤：

1. 搭建Gradle环境
2. 下载Spring源码
3. 导入源码到idea
4. 编译源码
5. 搭建测试项目
6. 启动调试
7. 查看Spring执行流程和源码

为什么要讲解Gradle？

从Spring5开始，官方就开始使用Gradle来构建环境了。所以接下来，我们的环境基于Gradle构建。因此有必要在开始之前先简答说一下大家不常用的Gradle。

4.1.1 什么是gradle？

Gradle是一个项目自动化构建工具。是Apache的一个基于Ant 和Maven的软件，用于项目的依赖管

理。



项目的构建经历了三个时代：

1. Apache Ant (2000 年左右)
2. Maven (2004 年)
3. Gradle (2012 年左右)

从Spring5.0开始等优秀的开源项目都将自己的项目从 Maven 迁移到了 Gradle。Google 官方 Android 开发的 IDE Android Studio 也默认使用 Gradle 构建。

4.1.2 源码环境搭建与编译

1) 安装Gradle环境

注意：下载Gradle建议使用4.3.1即可，不要下载太高的版本，否则后面在idea中编译的时候有些Spring的组件下载不了

```
1 | https://services.gradle.org/distributions/  
2 | https://gradle.org/releases/
```

Gradle安装非常简单，与配置JDK、Maven一样：将主目录配置到环境变量即可（设置JAVA_HOME和GRADLE_HOME）

然后cmd打开窗口，测试一下是否安装成功

```
1 | gradle -v
```

```
Microsoft Windows [版本 10.0.19044.2251]  
(c) Microsoft Corporation。保留所有权利。  
  
C:\Users\Administrator>gradle -v  
  
-----  
Gradle 4.3.1  
-----  
  
Build time: 2017-11-08 08:59:45 UTC  
Revision: e4f4804807ef7c2829da51877861ff06e07e006d  
  
Groovy: 2.4.12  
Ant: Apache Ant(TM) version 1.9.6 compiled on June 29 2015  
JVM: 1.8.0_251 (Oracle Corporation 25.251-b08)  
OS: Windows 10 10.0 amd64
```

常见错误：Build failed with an exception

如果不能正常查看版本号，可能是你JDK版本太高，将JAVA_HOME修改为 8 再试试

```
C:\Users\admin>gradle -v

FAILURE: Build failed with an exception.

* What went wrong:
Could not determine java version from '11'.

* Try:
Run with --stacktrace option to get the stack trace. Run with --info or --debug option to get more log output.
```

2) 配置镜像加速

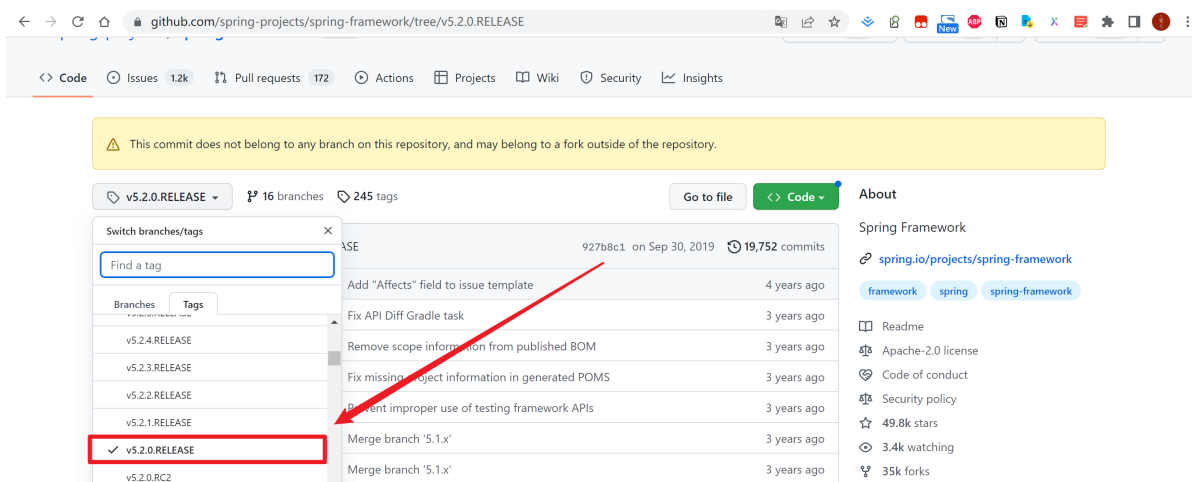
在~/.gradle/下创建 init.gradle文件，内容如下：(国内阿里云加速)

```
1 allprojects {
2     repositories {
3         //mavenLocal()
4         maven { url 'https://maven.aliyun.com/repository/central' }
5         maven { url 'https://maven.aliyun.com/repository/jcenter' }
6         maven { url 'https://maven.aliyun.com/repository/public' }
7         maven { url 'https://maven.aliyun.com/repository/google' }
8         maven { url 'https://maven.aliyun.com/repository/gradle-plugin' }
9         maven { url 'https://maven.aliyun.com/repository/grails-core' }
10        maven { url 'https://maven.aliyun.com/repository/spring' }
11        maven { url 'https://maven.aliyun.com/repository/spring-plugin' }
12        maven { url 'https://maven.aliyun.com/repository/apache-snapshots' }
13        mavenCentral()
14    }
15 }
```

3) 下载Spring源码

从Github下载Spring源码

```
1 | https://github.com/spring-projects/spring-framework.git
```



4) 源码环境编译

```
1 | gradlew.bat
```

执行项目下的gradlew.bat，等待构建完成...


```
D:\hero-workspace\spring-framework-5.2.0.RELEASE>gradlew.bat
Downloading https://services.gradle.org/distributions/gradle-5.6.2-bin.zip
.....

Welcome to Gradle 5.6.2!

Here are the highlights of this release:
- Incremental Groovy compilation
- Groovy compile avoidance
- Test fixtures for Java projects
- Manage plugin versions via settings script

For more details see https://docs.gradle.org/5.6.2/release-notes.html

Starting a Gradle Daemon (subsequent builds will be faster)
> Task :help

Welcome to Gradle 5.6.2.

To run a build, run gradlew <task> ...

To see a list of available tasks, run gradlew tasks

To see a list of command-line options, run gradlew --help

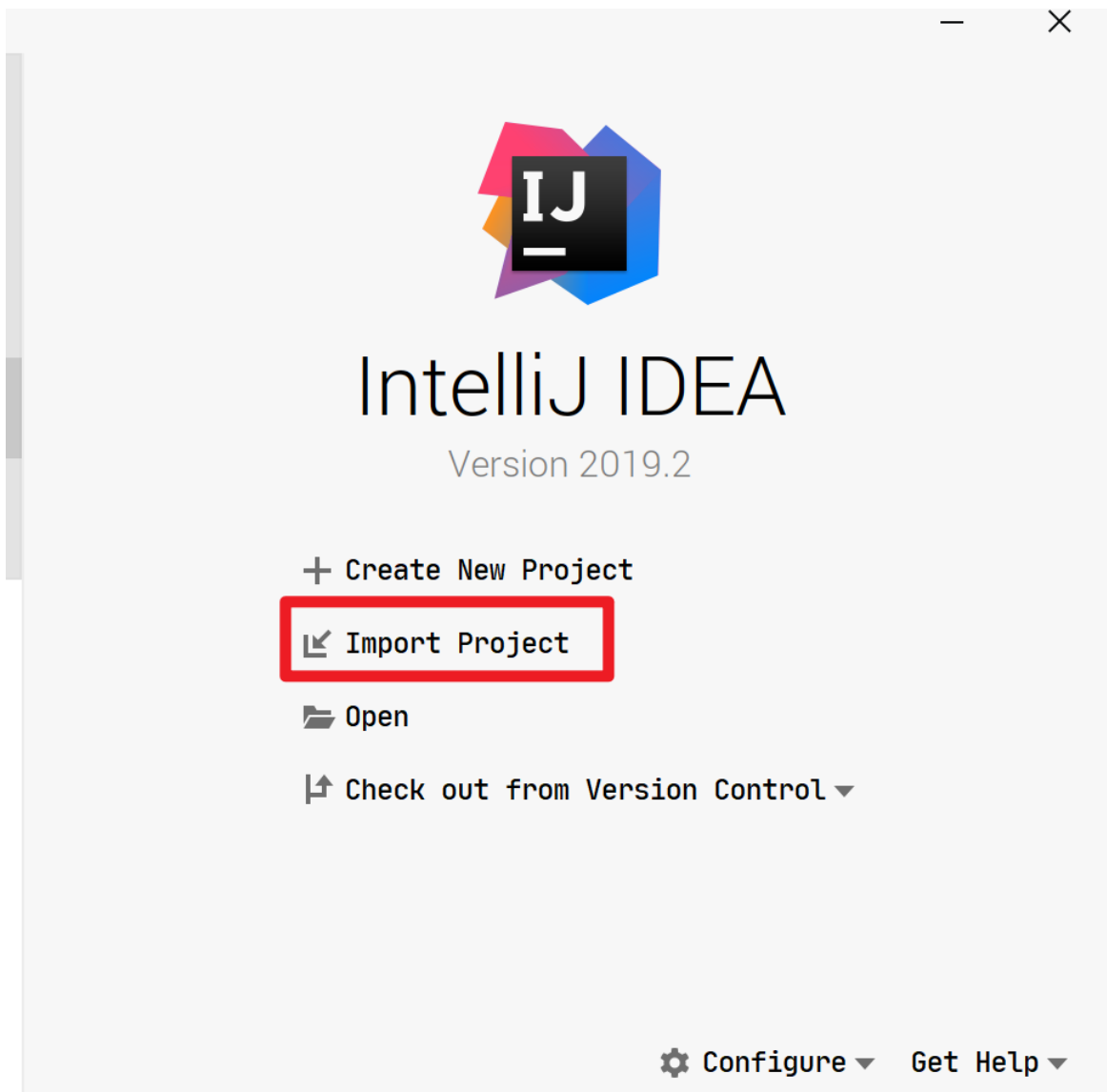
To see more detail about a task, run gradlew help --task <task>

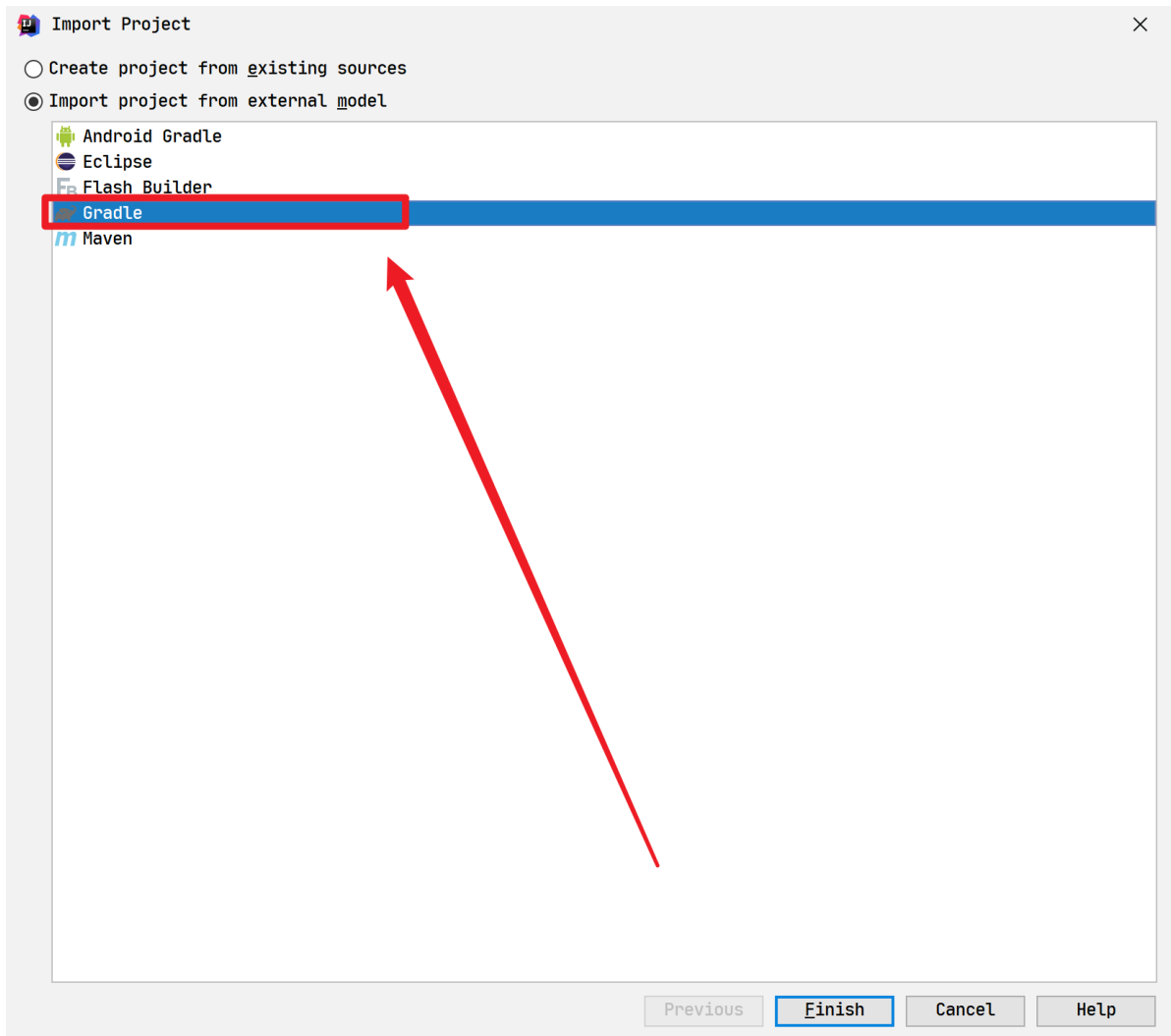
For troubleshooting, visit https://help.gradle.org

BUILD SUCCESSFUL in 2m 41s
1 actionable task: 1 executed
```

5) IDEA导入源码

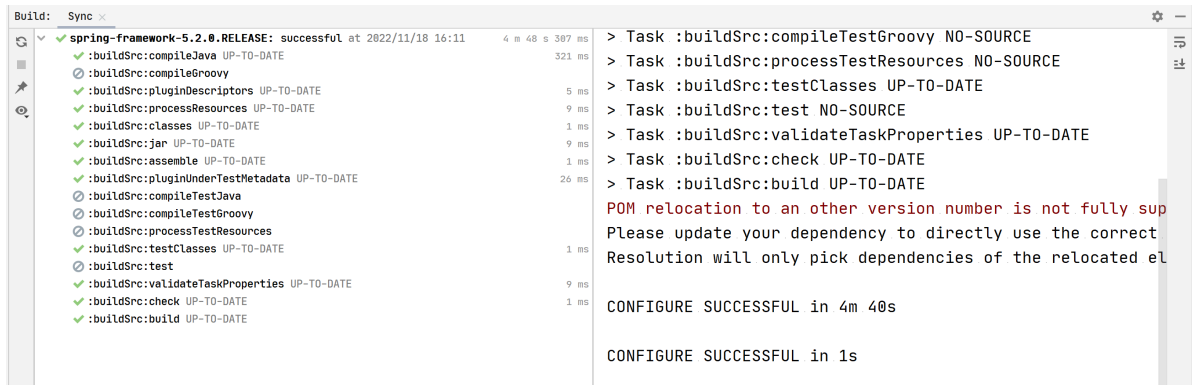
注意：这里必须使用import Project，不能使用open形式打开





然后点击 Finish 就会进入项目的自动构建阶段了。

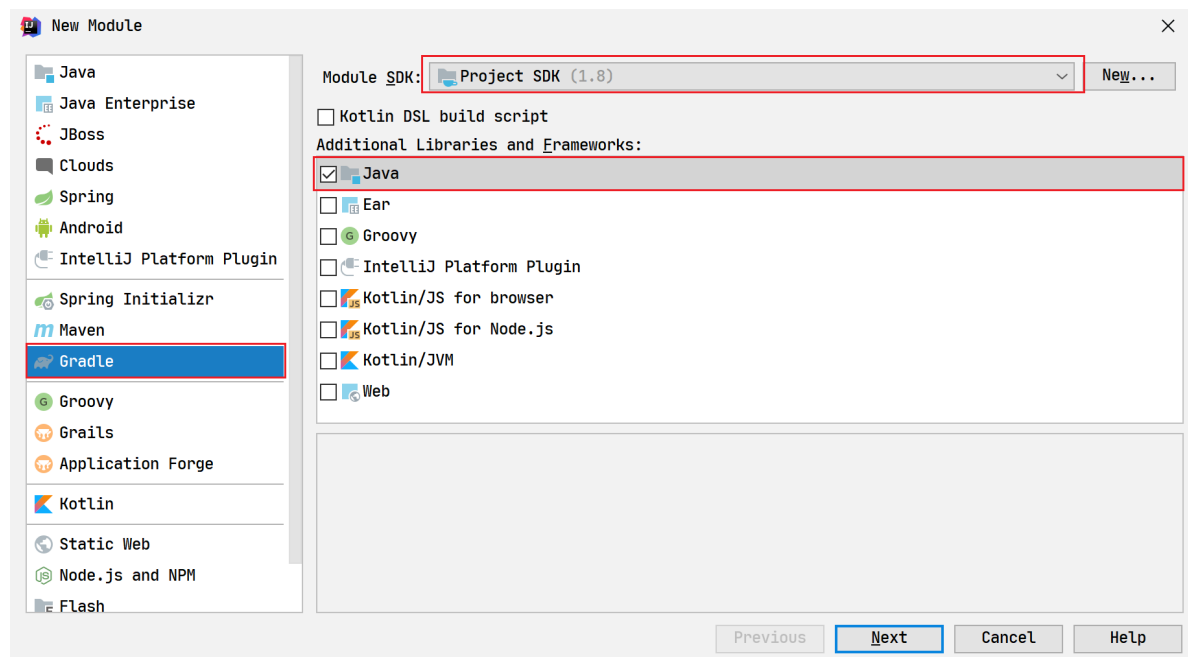
Spring 构建完成（耗时比较长，请耐心等待，可以去点一杯热咖啡了），如下图所示：正在编译，依据电脑配置的不同，预计执行4分多钟



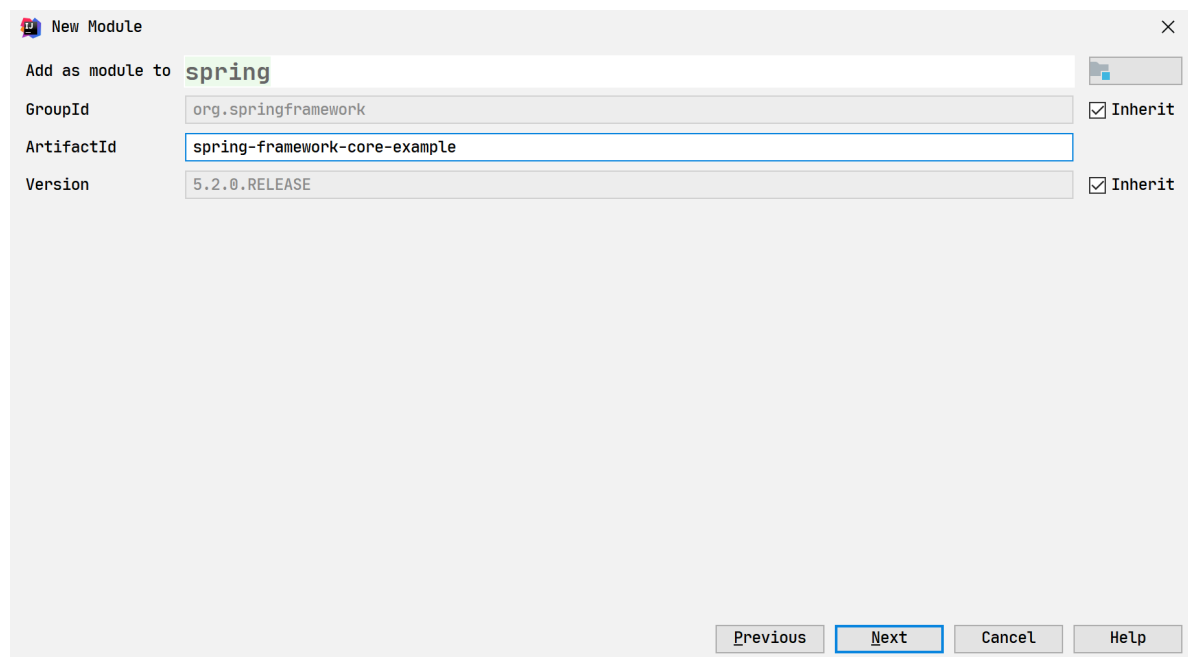
4.2 开始搭建测试项目

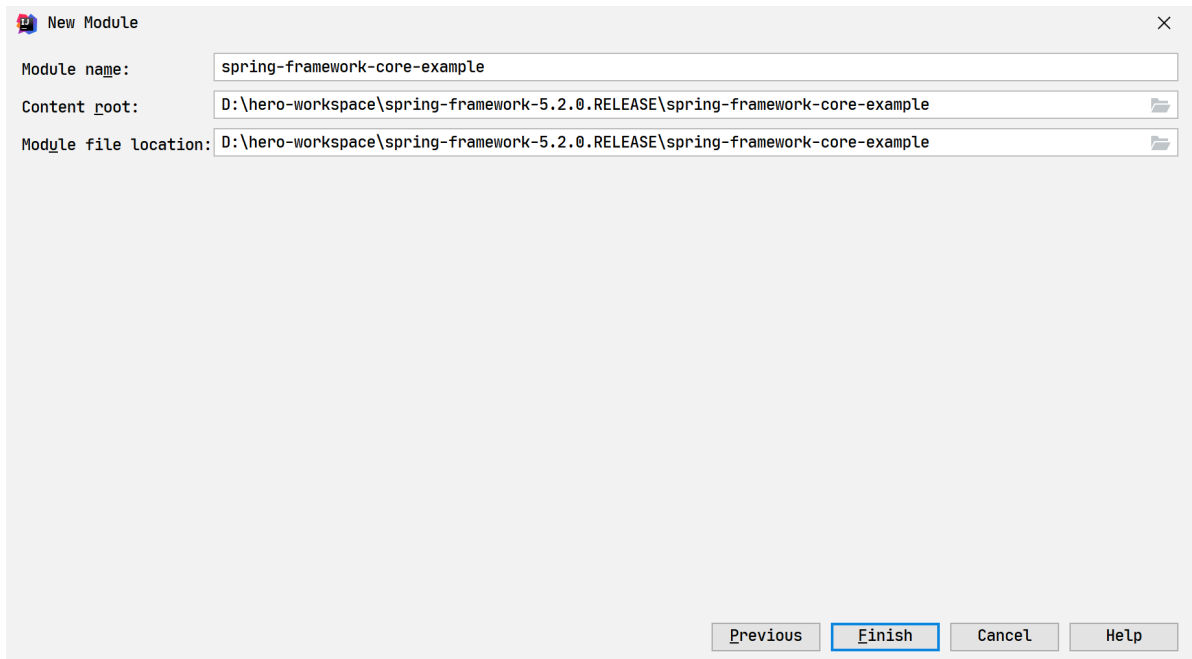
4.2.1 新建测试项目

首先我们在 Spring 源码项目中新增一个测试项目spring-framework-core-example，创建一个 Gradle 的 Java 项目



详细信息





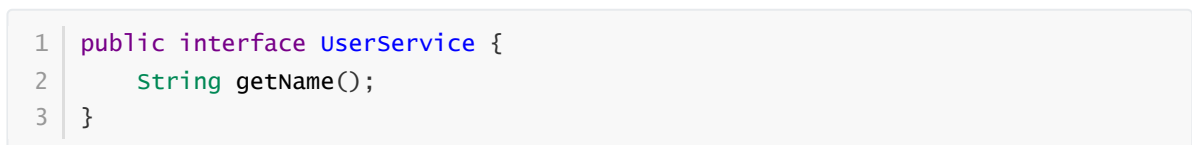
4.2.2 完善信息

在 build.gradle 中添加对 Spring 源码的依赖：



接着，我们需要在项目中创建一个 bean 和配置文件（applicationContext.xml）及启动文件（Main.java）

接口：



实现类：

```

1 public class UserServiceImpl implements UserService {
2     @Override
3     public String getName() {
4         return "Hello world";
5     }
6 }

```

TestIoC代码如下：

```

1 public class TestIoC {
2     public static void main(String[] args) {
3         ApplicationContext context =
4             new
5             ClassPathXmlApplicationContext("classpath*:applicationContext.xml");
6         UserService userService = context.getBean(UserService.class);
7         System.out.println(userService);
8         // 这句将输出: hello world
9         System.out.println(userService.getName());
10    }
11 }

```

配置文件 applicationContext.xml（在 resources 中）配置如下：

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xsi:schemaLocation="http://www.springframework.org/schema/beans
5                           http://www.springframework.org/schema/beans/spring-beans.xsd">
6
7     <bean id="userService" class="com.hero.service.impl.UserServiceImpl"/>
8 </beans>

```

运行：

```

1 输出如下
2 > Task :spring-framework-core-example:TestIoC.main()
3 com.hero.service.impl.UserServiceImpl@5c6648b0
4 Hello world

```

4.3 IoC初始化流程与继承关系

在看源码之前需要掌握Spring的继承关系和初始化

4.3.1 IoC容器初始化流程

目标：

- IoC容器初始化过程中到底都做了哪些事情【重点】
- IoC容器初始化是如何实例化Bean的【重点】

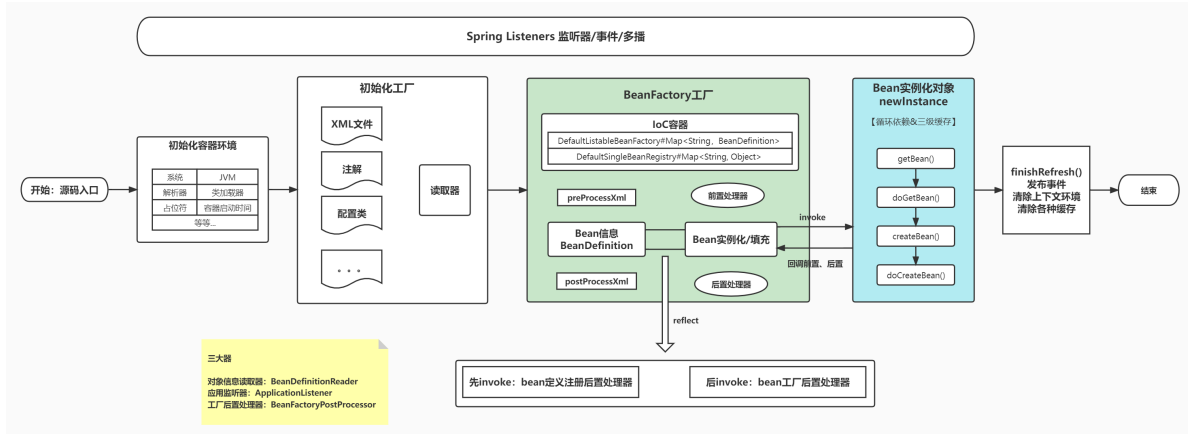
```

1 //没有Spring之前我们是这样的
2 User user=new User();
3 user.xxx();
4
5 //有了Spring之后我们是这样的
6 <bean id="userService" class="com.hero.service.impl.UserServiceImpl">
7 User user= applicationContext.getBean("xxx");
8 user.xxx();

```

1) IoC初始化流程简化图:

在剖析源码的整个过程中，我们一直会拿着这个图和源码对照



初始化:

- 容器环境的初始化
- Bean工厂的初始化: 销毁, 创建, 覆盖, 循环引用

读取与定义

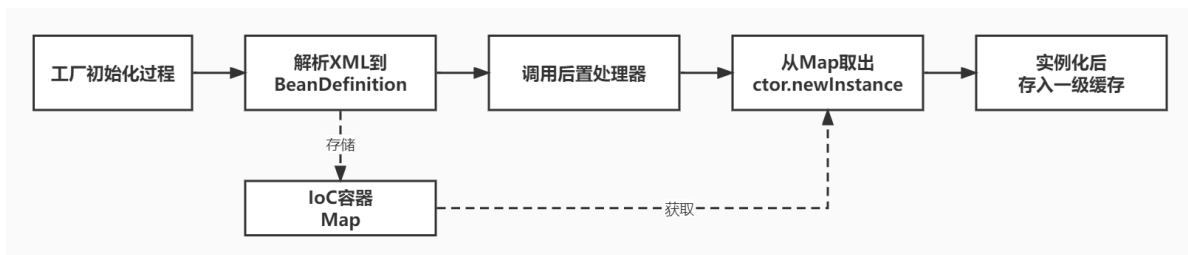
- 读取: 通过BeanDefinitionReader读取我们项目中的配置 (applicationContext.xml)
- 定义: 通过解析XML文件内容, 将里面的Bean解析成BeanDefinition (未实例化、未初始化)

实例化与销毁

- Bean实例化、初始化
- 销毁缓存等

2) IoC容器如何创建单例对象:

关注重点: ①单例Bean是如何实例化、②遇到循环引用如何处理

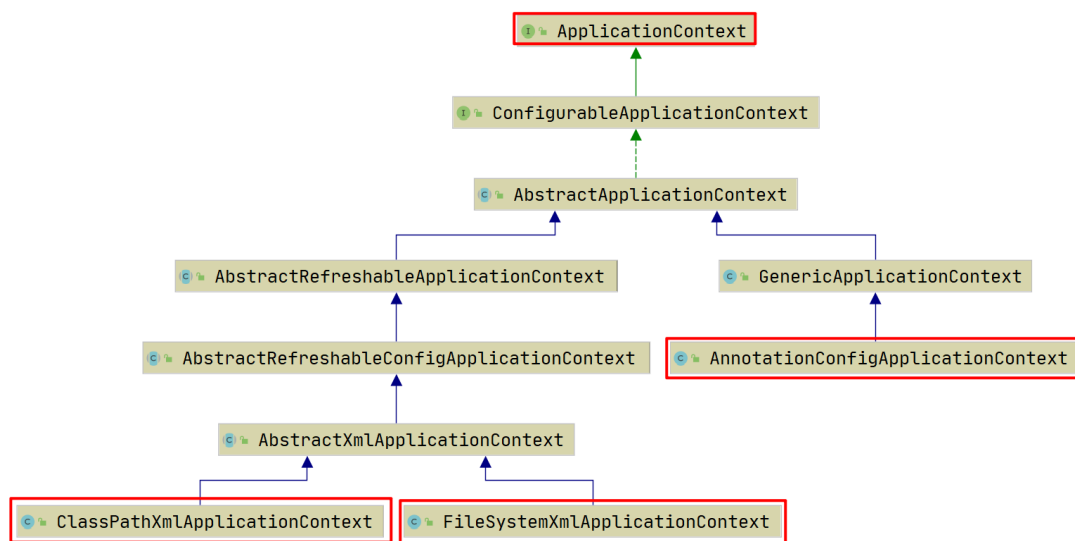


- 工厂初始化过程
- 解析XML到BeanDefinition放到Map
- 调用后置处理器
- 从Map取出进行实例化 (ctor.newInstance)

- 实例化后放到一级缓存

4.3.2 容器与工厂继承体系

容器构建继承关系

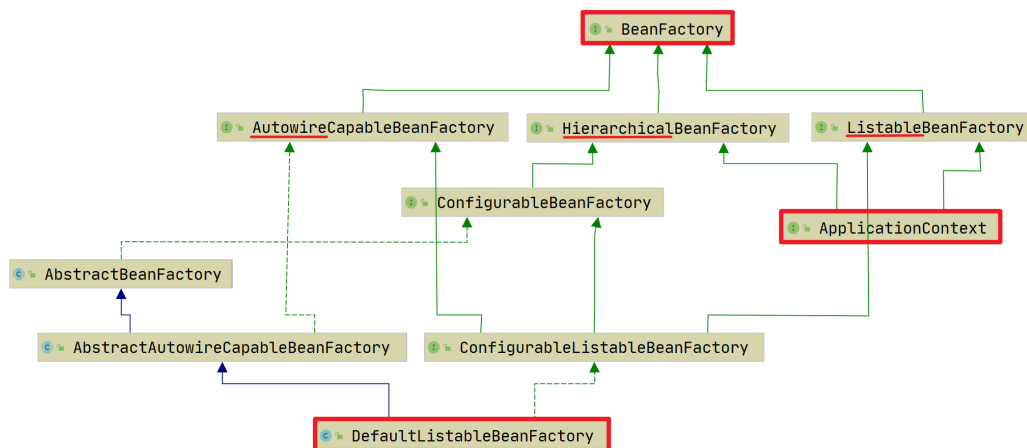


继承关系理解：

- **ClassPathXmlApplicationContext**的关键接口是 **ApplicationContext**，通过ClassPath加载XML配置，初始化容器
- **FileSystemXmlApplicationContext** 通过文件系统加载XML配置，初始化容器
- **AnnotationConfigApplicationContext** 通过加载注解，初始化容器

ApplicationContext 和 BeanFactory 啥关系？

接口的继承关系



4.4 工厂的构建过程【重点】

目标：IoC容器初始化过程中到底都做了哪些事情

总体流程：

```
1 ApplicationContext appContext = new
  ClassPathXmlApplicationContext("applicationContext.xml");
2 //ApplicationContext入口
3 appContext#构造方法()//
4     appContext#super(parent)//调用父类构造方法：初始化累加器，解析器，环境变量
5     AbstractXmlApplicationContext
6     AbstractRefreshableConfigApplicationContext
7     AbstractRefreshableApplicationContext
8     AbstractApplicationContext
9         getResourcePatternResolver();//获得解析器
10        setParent(parent);//设置父容器
11 appContext#setConfigLocations()//Placeholder路径解析器，将XML路径存储到数组
12 appContext#refresh()
13     AbstractApplicationContext#prepareRefresh();//预刷新
14     //校验系统环境属性
15
16 AbstractApplicationContext#getEnvironment().validateRequiredProperties();
17     AbstractApplicationContext#obtainFreshBeanFactory();//创建bean工厂
18     AbstractApplicationContext#refreshBeanFactory();//刷新bean工厂(也是在创
    建Bean工厂)
19     AbstractRefreshableApplicationContext#createBeanFactory();//创建
    bean工厂
20     AbstractRefreshableApplicationContext#loadBeanDefinitions();//解
    析XML配置文件
21     AbstractXmlApplicationContext#loadBeanDefinitions();//加载配
    置文件并解析
22     XmlBeanDefinitionReader#loadBeanDefinitions();//解析配置文
    件，重载方法
23     XmlBeanDefinitionReader#doLoadBeanDefinitions();//执行解
    析
24     XmlBeanDefinitionReader#doLoadDocument();//XML输入流转换为
    DOM对象【重点】
25     XmlBeanDefinitionReader#registerBeanDefinitions();//注册
    BeanDefinition
26     BeanDefinitionDocumentReader#registerBeanDefinitions()
27     DefaultBeanDefinitionDocumentReader#registerBeanDefinitions()
28     DefaultBeanDefinitionDocumentReader#doRegisterBeanDefinitions();
29     DefaultBeanDefinitionDocumentReader#parseBeanDefinitions();
30     DefaultBeanDefinitionDocumentReader#parseDefaultElement();
31     DefaultBeanDefinitionDocumentReader#processBeanDefinition();
32     BeanDefinitionParserDelegate#parseBeanDefinitionElement();//end【重点】
    //创建新的 BeanFactory=DefaultListableBeanFactory
```



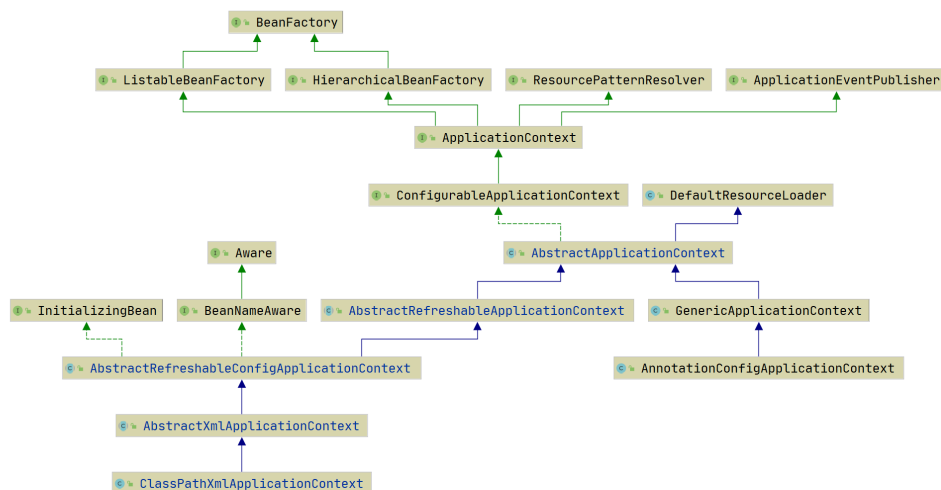
```

33 //解析XML，注册Bean的XML定义信息，此时还未进行对象创建
34 AbstractApplicationContext#prepareBeanFactory();//工厂前置操作
35 //设置：类加载器、表达式解析器、属性编辑器...
36 AbstractApplicationContext#postProcessBeanFactory();//工厂后置操作
37 AbstractApplicationContext#invokeBeanFactoryPostProcessors();//调用对
象工厂后置处理器
38 AbstractApplicationContext#registerBeanPostProcessors();//注册对象后置
处理器
39 AbstractApplicationContext#initMessageSource();//初始化消息源，解决国际
化问题
40 AbstractApplicationContext#initApplicationEventMulticaster();//初始化
事件广播器，观察者设计模式
41 AbstractApplicationContext#onRefresh();//刷新
42 AbstractApplicationContext#registerListeners();//注册监听器
43
44 AbstractApplicationContext#finishBeanFactoryInitialization(beanFactory);//
完成对象工厂初始化操作，负责创建所有的 单例对象
AbstractApplicationContext#finishRefresh();//完成刷新

```

详细解析如下：

1) ApplicationContext入口



参考 locTest.java

```

1 //1.创建Spring的ioc容器，ApplicationContext，通过xml配置文件初始化
2 ApplicationContext ac = new
  ClassPathXmlApplicationContext("applicationContext.xml");
3 //2.配置XML配置文件applicationContext.xml
4 //3.从ioc容器中取出容器创建好的对象(bean)
5 AccountService accountService = ac.getBean(AccountService.class);
6 Account account = ac.getBean(Account.class);
7 //4.调用执行对象中的方法测试
8 accountService.save(account);

```

进入到ClassPathXmlApplicationContext的有参构造器ClassPathXmlApplication()

```

1 public ClassPathXmlApplicationContext( String[] c, boolean r,
  ApplicationContext p) throws Exception {
2     //继承结构图Ctrl + H
3     //调用父类构造方法
4     // 1、获得一个ClassLoader
5     // 2、创建一个解析器
6     // 3、获取环境（系统环境、jvm环境）
7     super(parent);
8     // 4、设置Placeholder占位符解析器
9     // 5、将xml的路径解析完存储到数组
10    setConfigLocations(configLocations);
11    if (refresh) {
12        refresh();
13    }
14 }

```

解析

- super()做的事情：
 1. 父子容器
 2. 调用到顶端一共 5 层
 3. 初始化类加载器和解析器
 4. 获取的系统环境和jvm环境
- setConfigLocations()做的事情：
 1. 路径的解析
 2. 设置占位符解析器
- **进入核心方法refresh()**

2) 预刷新

prepareRefresh()

```

1 // synchronized锁
2 // 不然 refresh() 还没结束，又来一个启动或销毁容器的操作可就不妙了
3 synchronized (this.startupShutdownMonitor) {
4     //1、准备刷新
5     prepareRefresh();

```

解析

prepareRefresh做的事情

1. 记录启动时间/设置开始标志
2. 子类属性扩展（模板方法）
3. 校验系统环境属性
4. 初始化早期发布的应用程序事件监听对象

3) 创建bean工厂

obtainFreshBeanFactory()

获得新的bean工厂

目的：解析XML，注册Bean的XML定义信息

```
1 //获取并刷新BeanFactory
2 ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();
```

obtainFreshBeanFactory()做的事情

- 关闭旧的 BeanFactory
- 创建新的 BeanFactory=DefaultListableBeanFactory
- 解析XML，加载Bean的XML定义信息，注册Bean定义信息到BeanFactory（此时还未进行对象创建）
- 返回新的BeanFactory

4) 工厂前置操作

prepareBeanFactory()

```
1 prepareBeanFactory(beanFactory);
```

prepareBeanFactory()做的事情

1. 设置 BeanFactory 的类加载器
2. 设置 BeanFactory 的表达式解析器
3. 设置 BeanFactory 的属性编辑器
4. ...

5) 工厂后置操作

postProcessBeanFactory() 空方法

6) 工厂后置处理器

invokeBeanFactoryPostProcessors(beanFactory);

调用bean工厂后置处理器

7) 注册对象后置处理器

registerBeanPostProcessors(beanFactory);

注册bean后置处理器

```
1 //注册bean后置处理器，只是注册，但是不会反射调用。找出所有实现BeanPostProcessor接口的类
2 registerBeanPostProcessors(beanFactory)
```

查看重要的3步：最终目的都是实现对象后置处理器的注册

- 第一步：implement PriorityOrdered
- 第二步：implement Ordered.
- 第三步：Register all internal BeanPostProcessors.

8) 国际化

```
i18n initMessageSource();
```

初始化消息源，解决国际化问题，非核心步骤，跳过

9) 初始化事件广播器

```
initApplicationEventMulticaster();
```

初始化应用程序事件多路广播，观察者设计模式

10) 刷新

```
onRefresh();
```

刷新，方法是空的，交给子类实现：默认情况下不执行任何操作

```
1 // 具体的子类可以在这里初始化一些特殊的 Bean（在初始化 singleton beans 之前）
2 onRefresh(); //不重要，跳过
```

11) 注册监听器

```
registerListeners();
```

注册所有监听器

11) 完成BeanFactory

```
finishBeanFactoryInitialization(beanFactory);
```

完成bean工厂初始化操作

```
1 //完成bean工厂初始化操作,负责创建所有的 单例对象
2 //此处开始调用对象的前置处理器和后置处理器
3 finishBeanFactoryInitialization(beanFactory);
```

1. 设置处理器如：解析器、转换器、类装载器
2. 实例化
3. 填充

13) 完成刷新

- 发布事件
- 清楚上下文环境
- 清楚各种缓存

```
1  protected void finishRefresh() {
2      // 1、清除上下文级资源缓存
3      clearResourceCaches();
4      // 2、LifecycleProcessor初始化
5      // ps: 当ApplicationContext启动或停止时，它会通过LifecycleProcessor来与所有
      声明的bean的周期做状态更新
6      // 而在LifecycleProcessor的使用前首先需要初始化
7      initLifecycleProcessor();
8      // 3、启动所有实现了LifecycleProcessor接口的bean
9      // 默认实现DefaultLifecycleProcessor
10     getLifecycleProcessor().onRefresh();
11     // 4、发布上下文刷新完毕事件到相应的监听器
12     // 当完成容器初始化的时候，要通过Spring中的事件发布机制来发出
      ContextRefreshedEvent事件，以保证对应的监听器可以做进一步的逻辑处理
13     publishEvent(new ContextRefreshedEvent(this));
14     // 5、把当前容器注册到到MBeanServer，用于jmx使用
15     LiveBeansView.registerApplicationContext(this);
16 }
```

4.5 Spring 单例对象创建【重点】

目标：IoC容器初始化是如何实例化Bean

参考代码：先看没有循环依赖的情况，普通单例bean的初始化 SingleBeanInitTest.java

1) 调用入口

大家都知道是getBean()方法，但是这个方法要注意，有很多调用时机。

如果你把断点打在了这里，再点进去getBean，你将会直接从singleton集合中拿到一个实例化好的bean无法看到它的实例化过程。

可以debug试一下。

会发现接从getSingleton返回了bean，这不是我们想要的模样.....

```
public class SingleBeanInitTest {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("classpath*:applicationContext.xml");
        UserService userService = context.getBean(UserService.class);
        System.out.println(userService);
        // 这句将输出: hello world
        System.out.println(userService.getName());
    }
}
```

思考一下，为什么呢？

- 在对象工厂完成后，会对singleton的bean完成初始化，那么真正的初始化应该发生在那里？
- 那就需要找到：DefaultListableBeanFactory的第 874行的getBean



```
1 ApplicationContext appContext = new
  ClassPathXmlApplicationContext("applicationContext.xml");
2 //ApplicationContext入口
3 appContext#构造方法()//
4   appContext#super(parent)//调用父类构造方法：初始化累加器，解析器，环境变量
5   appContext#setConfigLocations()//Placeholder路径解析器，将XML路径存储到数组
6   appContext#refresh()
7   //...
8   //完成对象工厂初始化操作，负责创建所有的 单例对象
9   AbstractApplicationContext#finishBeanFactoryInitialization(beanFactory);
10   DefaultListableBeanFactory#preInstantiateSingletons();
11   AbstractBeanFactory#getBean()//获取Bean
12   AbstractBeanFactory#doGetBean()//执行获取操作
13   //先从缓存拿，没有则创建一个新的
14   DefaultSingletonBeanRegistry#getSingleton(beanName)
15   //函数式接口传参，在参数中执行createBean()方法
16
17   DefaultSingletonBeanRegistry#getSingleton(beanName,singletonFactory)
18   AbstractAutowireCapableBeanFactory#createBean()//创建Bean
19   AbstractAutowireCapableBeanFactory#doCreateBean()//执行创建操
20   AbstractAutowireCapableBeanFactory#createBeanInstance()//创建
21   AbstractAutowireCapableBeanFactory#instantiateBean()//使
22   SimpleInstantiationStrategy#instantiate()//实例化
23   BeanUtils#instantiateClass(constructorToUse);//
24   ctor#newInstance(argsWithDefaultValues);//反射创
25   // 清除2、3级，放入1级
26   DefaultSingletonBeanRegistry#addSingleton(beanName,
27   singletonObject);
28   AbstractAutowireCapableBeanFactory#populateBean()//注入属性
29   AbstractAutowireCapableBeanFactory#initializeBean()//初始化给
30   AbstractAutowireCapableBeanFactory#invokeAwareMethods()
31   AbstractAutowireCapableBeanFactory#applyBeanPostProcessorsBeforeInitializat
32   ion()
33   AbstractAutowireCapableBeanFactory#invokeInitMethods()
```

```
AbstractAutowireCapableBeanFactory#applyBeanPostProcessorsAfterInitialization()
```

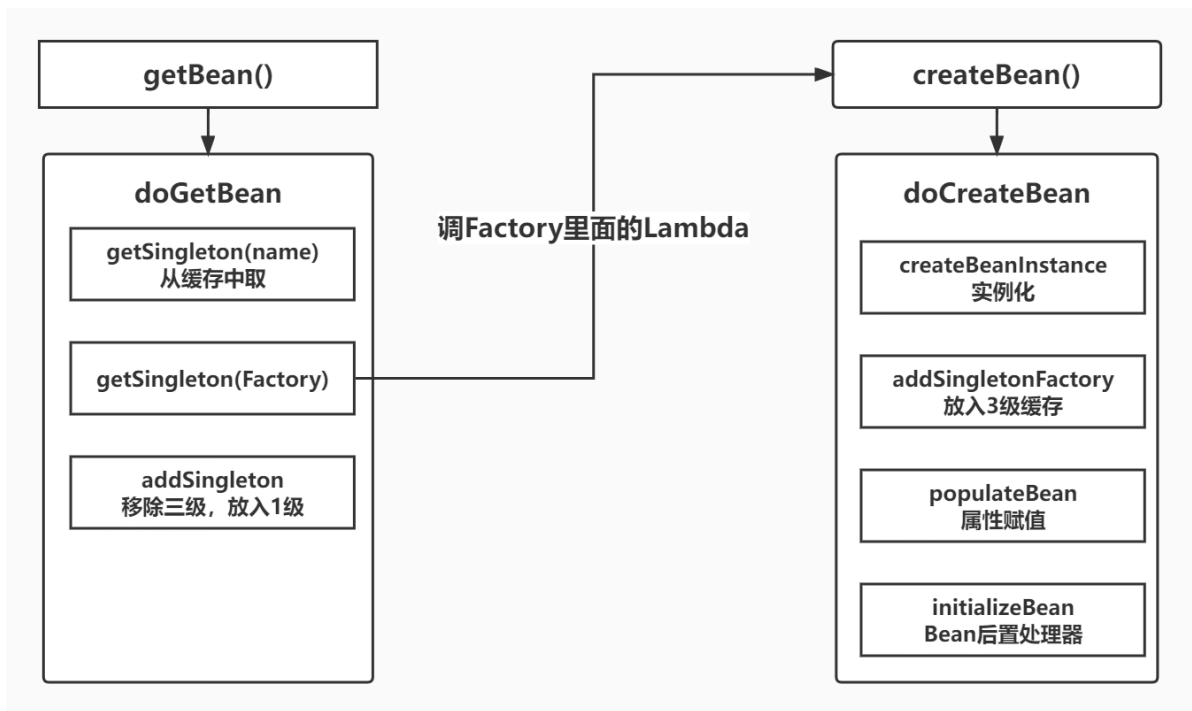
2) 主流程

先搞清除 3 级缓存，关于bean的三级缓存：DefaultSingletonBeanRegistry

```
1  /**
2   * 一级缓存：单例对象缓存池，存放的 Bean 已经实例化、属性赋值、完全初始化好（成品）
3   * 它可能来自3级，或者2级
4   */
5   private final Map<String, Object> singletonObjects = new
ConcurrentHashMap<>(256);
6   /**
7   * 二级缓存：早期单例对象缓存池，存放的 Bean 已经实例化但尚未属性赋值、未执行 `init` 方法（半成品）
8   */
9   private final Map<String, Object> earlySingletonObjects = new HashMap<>
(16);
10  /**
11  * 三级缓存：单例工厂缓存池，这里面不是bean本身，是它的一个工厂，未来调getObject来获取真正的bean
12  */
13  private final Map<String, ObjectFactory<?>> singletonFactories = new
HashMap<>(16);
```

缓存层级	名称	描述
第一层缓存	<code>singletonObjects</code>	单例对象缓存池，存放的 Bean 已经实例化、属性赋值、完全初始化好（成品）
第二层缓存	<code>earlySingletonObjects</code>	早期单例对象缓存池，存放的 Bean 已经实例化但尚未属性赋值、未执行 <code>init</code> 方法（半成品）
第三层缓存	<code>singletonFactories</code>	单例工厂的缓存，这里面不是bean本身，是它的一个工厂，未来调getObject来获取真正的bean

主流程图很重要！后面的debug会带着这张图走



- `getBean`: 入口
- `doGetBean`: 调`getSingleton`查一下缓存看看有没有, 有就返回, 没有给`Singleton`一个`Lambda`表达式, 函数里调下面的`createBean`拿到新的`bean`, 然后清除3级缓存, 放入1级缓存
 - `createBean`: 到这里, 一堆检查后, 进入下面真正创建`bean`的地方
 - `doCreateBean`: 调构造函数初始化, 放入3级缓存, 解析属性赋值, 调用`Bean`后置处理器

3) `getSingleton`

在`DefaultSingletonBeanRegistry`里, 有三个, 作用完全不一样

```

1 //啥也没干, 调下面传了个true
2 public Object getSingleton(String beanName)
3
4 //从1级缓存拿, 如果没有且参数是true, 就使用3级升2级返回, 否则返回null
5 protected Object getSingleton(String beanName, boolean allowEarlyReference)
6
7 //从1级缓存拿, 如果没有通过给的factory创建并放入1级里
8 public Object getSingleton(String beanName, ObjectFactory<?>
   singletonFactory)

```

4) `bean`实例化

```

1 org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory#
   createBeanInstance

```


5) 放入三级缓存

```
1 | org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory#  
   | addSingletonFactory
```

6) 注入属性

```
1 | AbstractAutowireCapableBeanFactory#populateBean()//注入属性的地方
```

7) 初始化Bean

调用bean前后置处理器对应方法

```
1 | AbstractAutowireCapableBeanFactory#initializeBean()//初始化Bean，调用前后置处理器
```

Spring开始加载Bean

1. 首先实例化Bean

- 调用AbstractAutowireCapableBeanFactory#createBeanInstance方法实例化Bean
- 此时Bean只实例化，并未进行@Autowired的属性填充

2. 填充Bean属性

- 如果Bean属性有@Autowired，则需要进行属性填充
- 进行属性填充的前提是要保证属性实例已经存在Spring容器中，如果不存在，则会先去加载属性【三级缓存】

3. 初始化Bean

- 3.1 调用invokeAwareMethods方法
 - 实现BeanNameAware接口，调用setBeanName()方法
 - 实现BeanClassLoaderAware接口，调用setBeanClassLoader()方法
 - 实现BeanFactoryAware接口，调用setBeanFactory()方法
- 3.2 调用applyBeanPostProcessorsBeforeInitialization()方法
 - 循环调用实现了BeanPostProcessor接口的postProcessBeforeInitialization()方法，由于Spring自带ApplicationContextAwareProcessor类重写了postProcessBeforeInitialization()方法，所以会优先循环到此方法
 - 执行到ApplicationContextAwareProcessor#postProcessBeforeInitialization()方法，会检查是否实现Aware接口，这里重点关注Aware接口的ApplicationContextAware，如果实现ApplicationContextAware接口，则会调用setApplicationContext方法
 - 在循环调用自定义实现的BeanPostProcessor接口，调用postProcessBeforeInitialization方法
- 3.3 调用invokeInitMethods()方法
 - 实现InitializingBean接口，调用afterPropertiesSet方法
 - 指定init-method方法，调用init-method()方法
- 3.4 调用applyBeanPostProcessorsAfterInitialization()方法
 - 循环调用实现类BeanPostProcessor接口的postProcessAfterInitialization()方法

小结

伪代码，无循环依赖时，生成bean流程一览

```
1  getBean("A") {
2      doGetBean("A") {
3          a = getSingleton("A") {
4              a = singletonObjects(); //查1级缓存, null
5              if ("创建过3级缓存") { //不成立
6                  //忽略
7              }
8              return a;
9          }
10         // null
11         if (a == null) {
12             a = getSingleton("A", ObjectFactory of) {
13                 a = of.getObject() ->{ //lambda表达式
14                     createBean("A") {
15                         doCreateBean("A") {
16                             createBeanInstance("A"); // A 实例化
17                             addSingletonFactory("A"); // A 放入3级缓存
18                             populateBean("A"); // A 注入属性
19                             initializeBean("A"); // A 后置处理器
20                         } //end doCreateBean("A")
21                     } //end createBean("A")
22                 } // end lambda A
23                 addSingleton("A", a) // 清除2、3级，放入1级
24             } // end getSingleton("A",factory)
25         } // end if(a == null)
26         return a;
27     } //end doGetBean("A")
28 } //end getBean("A")
```

4.6 Spring循环依赖问题

在上面，我们剖析了bean实例化的整个过程，也就是我们的Bean他是单独存在的，和其他Bean没有交集和引用。而我们在业务开发中，肯定会有多个Bean相互引用的情况，也就是所谓的**循环依赖**。

4.6.1 什么是循环依赖？

通俗的讲就是N个Bean互相引用对方，最终形成**闭环**。



项目代码介绍如下（测试类入口：CircleTest.java）

配置文件

```
1  <!--循环依赖BeanA依赖BeanB -->
2  <bean id="userServiceImplA" class="com.hero.service.impl.UserServiceImplA">
3      <property name="userServiceImplB" ref="userServiceImplB"/>
4  </bean>
5  <!--循环依赖BeanB依赖BeanA -->
6  <bean id="userServiceImplB" class="com.hero.service.impl.UserServiceImplB">
7      <property name="userServiceImplA" ref="userServiceImplA"/>
8  </bean>
```

userServiceImplA代码如下

```
1  public class UserServiceImplA implements UserService {
2      private UserServiceImplB userServiceImplB;
3
4      public void setUserServiceImplB(UserServiceImplB userServiceImplB) {
5          this.userServiceImplB = userServiceImplB;
6      }
7
8      @Override
9      public String getName() {
10         return "在UserServiceImplA的Bean中" + "userServiceImplB注入成功
>>>>>>>>" + userServiceImplB;
11     }
12 }
```

userServiceImplB代码如下

```
1  public class UserServiceImplB implements UserService {
2      private UserServiceImplA userServiceImplA;
3
4      public void setUserServiceImplA(UserServiceImplA userServiceImplA) {
5          this.userServiceImplA = userServiceImplA;
6      }
7
8      @Override
9      public String getName() {
10         return "在UserServiceImplB的Bean中" + "userServiceImplA注入成功
>>>>>>>>" + userServiceImplA;
11     }
12 }
13
```

入口Main

```

1 ApplicationContext context = new
  ClassPathXmlApplicationContext("classpath*:applicationContext.xml");
2
3 UserService userService =
  context.getBean("userServiceImplA",UserService.class);
4 System.out.println(userService.getName());

```

输出如下

```

十二月 04, 2020 2:06:08 下午 org.springframework.beans.factory.xml.XmlBeanDefinitionReader loadBeanDefinitions
信息: Loading XML bean definitions from URL [file:/E:/spring-framework-5.x/spring-code-test/out/production/clas
在UserServiceImplA的Bean中userServiceImplB注入成功>>>>>>>com.spring.test.impl.UserServiceImplB@1698c449

```

```
Process finished with exit code 0
```

4.6.2 Spring如何解决循环依赖

Spring 中使用了三级缓存的设计，来解决单例模式下的属性循环依赖问题。

注意：解决的只是单例模式下的set属性赋值的Bean属性循环依赖问题，对于多例Bean的和构造方法注入参数的循环依赖问题，并不能使用三级缓存设计解决。

假如无法解决循环依赖

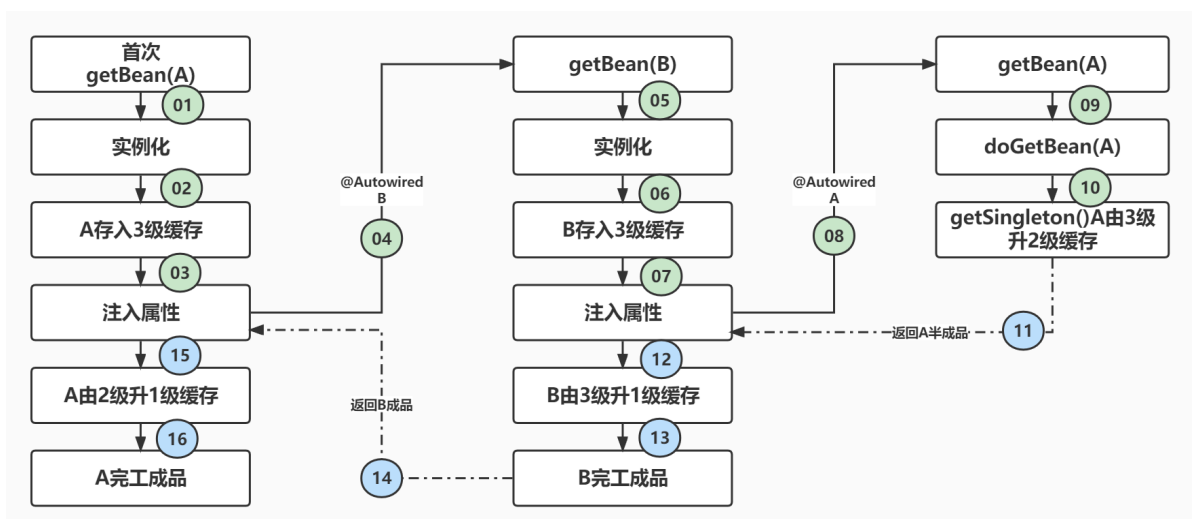
1. Bean无法成功注入，导致业务无法进行
2. 产生死循环

1) 三级缓存变化过程

目标：

只有明白三级缓存变化过程，才能知道是如何解决循环依赖的

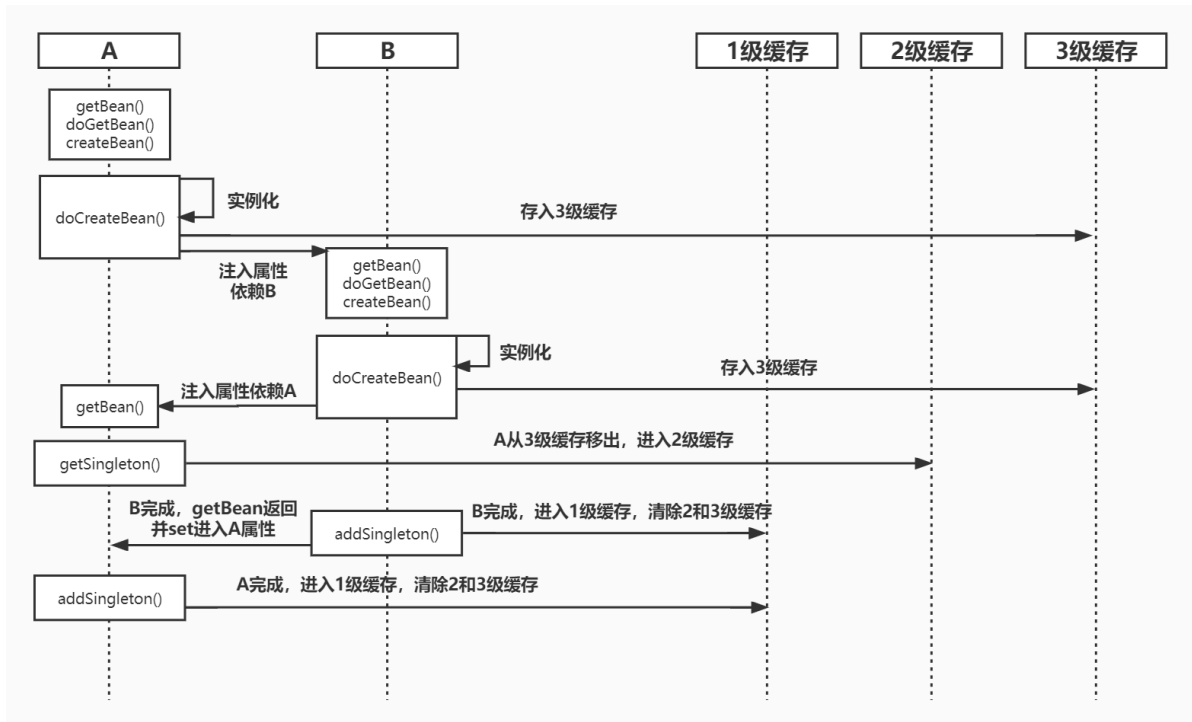
略去其他步骤，只看缓存变化



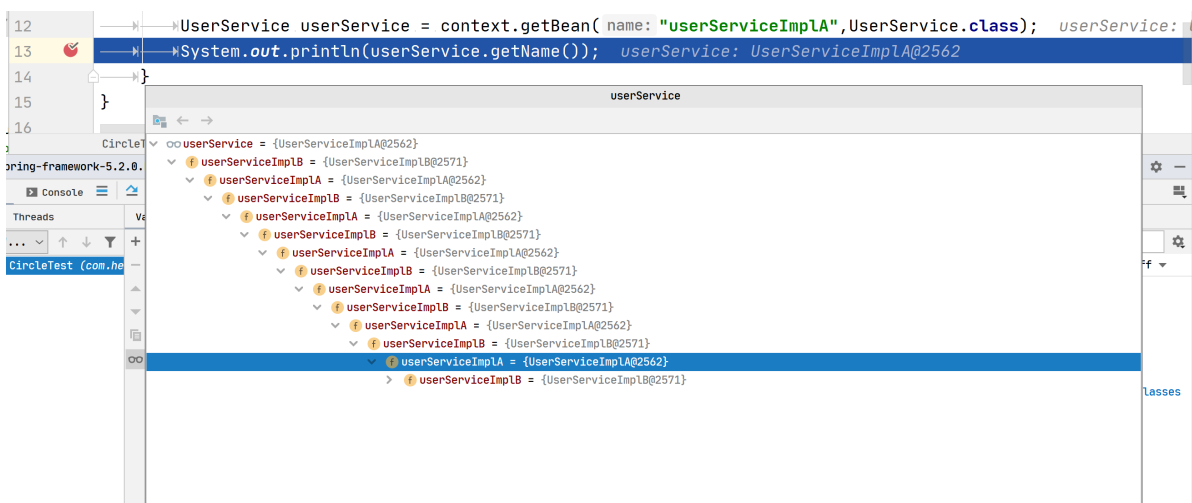
变化过程：

1. A实例化后、注入前放到3级缓存
2. B放到3级缓存后，A在注入属性时，发现有循环依赖，因此需要先getBean(B)，实例化B，并将B也从入3级缓存
3. B放到3级缓存后，这时B要开始注入属性A，于是B找到了循环依赖A后，再从头执行getBean(A)方法，getSingleton方法本次从缓存中取，然后将A设置到2级缓存，并且从3级缓存移除。
4. B如愿以偿的拿到了A完成注入，然后B执行到 DefaultSingletonBeanRegistry#addSingleton方法，将B从3级缓存移出，放入1级缓存，到此B完成。B的完成是被动的，A需要它，才会先去创建它
5. A 还要继续自己的流程，然后populateBean方法将B注入。然后，A移出2级缓存，进入1级缓存，整个流程完成！

时序图：



双方相互持有对方效果如下：



2) 三级缓存总结

伪代码，循环依赖流程一览，都是关键步骤

```

1  getBean("A") {
2      doGetBean("A") {
  
```

[illegible]

```

41      //移除3级缓存
42      singletonFactories.remove("A");
43      return a;
44  }
45  }
46  ; // A第二次, 不是null, 但是半成品, 还待在2级缓存里
47                                          } //
48  end doGetBean("A")
49                                          } // end
50  populate B
51      initializeBean("B", b); // B后置处理器
52                                          } // end
53  doCreateBean B
54                                          } // end createBean
55  B
56                                          } // end lambda B
57                                          // B 创建完成, 并且是完整
58 的, 虽然它里面的A还是半成品, 但不影响它进入1级
59                                          addSingleton("B", b); //
60 清除3级缓存, 进入1级
61                                          ); // end
62  getSingleton("B", factory)
63                                          } // end if(b==null);
64                                          return b;
65                                          } // end doGetBean("B")
66                                          } // end getBean("B")
67                                          } // end populateBean("A")
68                                          initializeBean("A"); // A 后置处理器
69                                          } //end doCreateBean("A")
70                                          } //end crateBean("A")
71                                          } // end lambda A
72                                          addSingleton("A", a) // 清除2、3级, 放入1级
73                                          } // end getSingleton("A", factory)
74                                          } // end if(a == null)
75                                          return a;
76                                          } //end doGetBean("A")
77  } //end getBean("A")

```

4.6.3 思考几个与循环依赖相关的小问题

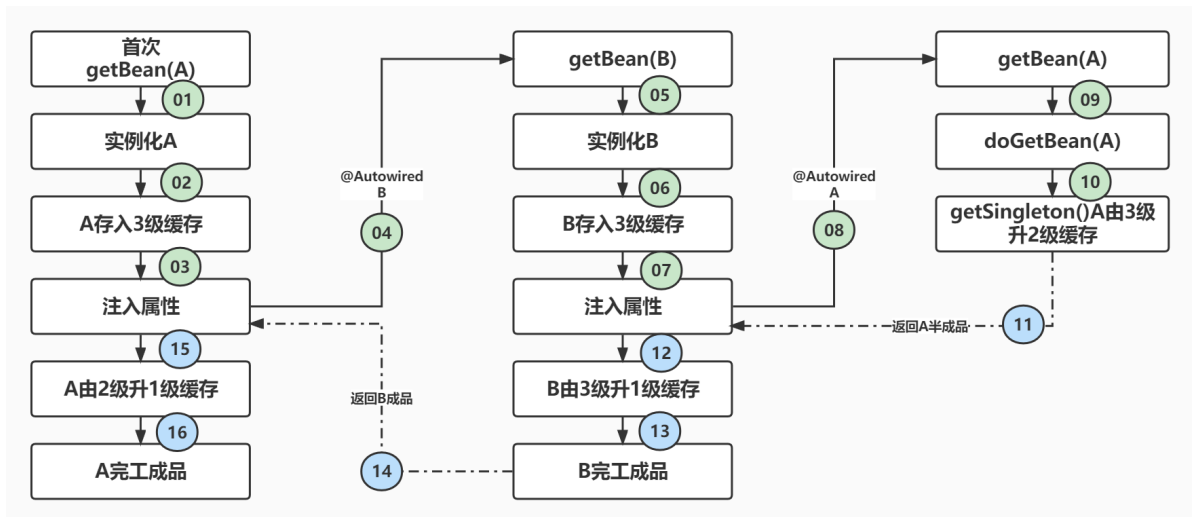
1) 为什么不能解决非单例、构造方法的循环依赖问题？

- 为什么不能解决构造方法的循环依赖？

- 对象构造函数会在实例化阶段调用

- 为什么不能解决多例的循环依赖？

- IoC容器只会管理单例Bean，并将单例Bean存入缓存。作用域为prototype时，每次getBean都会创建新的对象，并不存入缓存，因此不可以解决循环依赖问题。



2) 为什么设计三级？二级缓存能否解决循环依赖？

可以解决！

```
1 //修改三级缓存源码，不使用三级缓存
2 org.springframework.beans.factory.support.DefaultSingletonBeanRegistry#addSi
  ngletonFactory()
3 protected void addSingletonFactory(String beanName, ObjectFactory<?>
  singletonFactory) {
4     Assert.notNull(singletonFactory, "Singleton factory must not be null");
5
6     synchronized (this.singletonObjects) {
7         // 判断一级缓存中不存在此对象
8         if (!this.singletonObjects.containsKey(beanName)) {
9             // 直接从工厂中获取 Bean
10            Object o = singletonFactory.getObject();
11            // 不存入三级缓存，直接进入二级
12            // 添加至二级缓存中
13            this.earlySingletonObjects.put(beanName, o);
14            this.registeredSingletons.add(beanName);
15        }
16    }
17 }
```

使用三级而非二级缓存并非出于 IOC 的考虑，而是出于 AOP 的考虑，即若使用二级缓存，在 AOP 情形注入到其他 Bean 的，不是最终的代理对象，而是原始对象。

来咱们上源码来看一下：

AbstractAutowireCapableBeanFactory#getEarlyBeanReference()

```
1  if (!mbd.isSynthetic() && hasInstantiationAwareBeanPostProcessors()) {
2      //循环所有Bean后置处理器
3      for (BeanPostProcessor bp : getBeanPostProcessors()) {
4          if (bp instanceof SmartInstantiationAwareBeanPostProcessor) {
5              SmartInstantiationAwareBeanPostProcessor ibp =
6                  (SmartInstantiationAwareBeanPostProcessor) bp;
7              //开始创建AOP代理【重点】
8              exposedObject = ibp.getEarlyBeanReference(exposedObject,
9                  beanName);
10         }
11     }
12 }
13 //获取AOP代理对象的执行流程
14 AbstractAutowireCapableBeanFactory#getEarlyBeanReference()
15 AbstractAutoProxyCreator#getEarlyBeanReference()//获取代理对象
16 AbstractAutoProxyCreator#wrapIfNecessary()//获取代理对象
17 AbstractAutoProxyCreator#createProxy()//创建代理对象
18 ProxyCreatorSupport#createAopProxy()//创建Aop对应的代理对象创建工厂
19 DefaultAopProxyFactory#createAopProxy()//创建Aop代理对象
20 return new JdkDynamicAopProxy(config);//创建JDK的动态代理封装对象
21 ProxyFactory#getProxy()//获取代理对象
22 JdkDynamicAopProxy#getProxy()
23 Proxy.newProxyInstance(classLoader, proxiedInterfaces,
24     this);//JDK动态代理
25 CglibAopProxy#getProxy()
26 enhancer.create(constructorArgTypes,
27     constructorArgs);//cglib动态代理
```

总结下：

- 如果不调用后置处理器，返回的Bean和三级缓存一样，都是实例化、普通的Bean
- 如果调用后置，返回的就是代理对象，不是普通的Bean了
- 其实；这就是三级缓存设计的巧妙之处

5. Spring用到的设计模式

5.1 工厂模式

工厂模式 (Factory Pattern) 提供了一种创建对象的最佳方式

工厂模式 (Factory Pattern) 分为三种

- 简单工厂
- 工厂方法
- 抽象工厂

1. 简单工厂模式

```

1 ApplicationContext context =new
  ClassPathXmlApplicationContext("classpath*:applicationContext.xml");
2 UserService userService = context.getBean(UserService.class);

```

简单工厂模式对对象创建管理方式最为简单，因为其仅仅简单的对不同类对象的创建进行了一层简单的封装

定义接口IPhone

```

1 public interface Phone {
2     void make ();
3 }

```

实现类

```

1 public class IPHONE implements Phone {
2     public IPHONE() {
3         this .make ();
4     }
5     @Override
6     public void make () {
7         System.out.println("生产苹果手机!");
8     }
9 }

```

实现类

```

1 public class MiPhone implements Phone {
2     public MiPhone() {
3         this .make ();
4     }
5     @Override
6     public void make () {
7         System.out.println("生产小米手机!");
8     }
9 }

```

定义工厂类并且测试

```

1 public class PhoneFactory {
2     public Phone makePhone(String phoneType) {
3         if (phoneType.equalsIgnoreCase("MiPhone")) {
4             return new MiPhone();
5         } else if ( phoneType.equalsIgnoreCase("iPhone")) {
6             return new IPHONE();
7         }
8         return null;
9     }
10    public static void main (String[] arg ) {
11        PhoneFactory factory = new PhoneFactory();
12        Phone miPhone = factory.makePhone("MiPhone");
13        System.out.println("miPhone = " + miPhone);
14        IPHONE iPHONE = (IPHONE) factory.makePhone("iPHONE");

```

```
15         System.out.println("iPhone = " + iPhone);
16     }
17 }
```

5.2 模板模式

模板模式(Template Pattern):基于抽象类的, 核心是封装算法

```
1 Spring核心方法refresh就是典型的模板方法
2 org.springframework.context.support.AbstractApplicationContext#refresh
```

模板方法定义了一个算法的步骤，并允许子类为一个或多个步骤提供具体实现

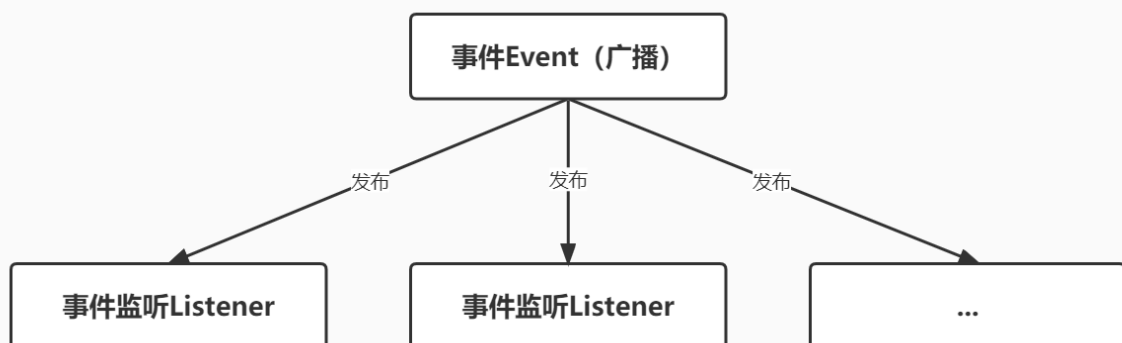
```
1 //模板模式
2 public abstract class TemplatePattern {
3     protected abstract void step1();
4     protected abstract void step2();
5     protected abstract void step3();
6     protected abstract void step4();
7     //模板方法
8     public final void refresh() {
9         //此处也可加入当前类的一个方法实现，例如init()
10        step1();
11        step2();
12        step3();
13        step4();
14    }
15 }
```

定义子类

```
1 //模板模式
2 public class SubTemplatePattern extends TemplatePattern {
3     //测试
4     public static void main(String[] args) {
5         TemplatePattern tp = new SubTemplatePattern();
6         tp.refresh();
7     }
8
9     @Override
10    public void step1() {
11        System.out.println(">>>>>>>>>>1");
12    }
13
14    @Override
15    public void step2() {
16        System.out.println(">>>>>>>>>>2");
17    }
18
19    @Override
20    public void step3() {
21        System.out.println(">>>>>>>>>>3");
```

输出

- 1.把冰箱门打开
- 2.把大象缩小
- 3.把大象放进冰箱
- 4.把冰箱门关上



案例：使用Spring事件监听机制完成事件、监听、发布流程

1) 定义事件

自定义一个事件MessageSourceEvent并且实现ApplicationEvent接口

```
1 //在Spring 中使用事件监听机制(事件、监听、发布)
2 //定义事件
3 public class MessageSourceEvent extends ApplicationEvent {
4     public MessageSourceEvent(Object source) {
5         super(source);
6         System.out.println("进入到事件源的有参数构造器");
7     }
8     void print() {
9         System.out.println("进入到事件源的方法");
10    }
11 }
```

2) 定义监听

有了事件之后还需要自定义一个监听用来接收监听到事件，自定义ApplicationContextListener监听 需要交给Spring容器管理， 实现ApplicationListener接口并且重写onApplicationEvent方法。

监听一

```
1 //在Spring中使用事件监听机制(事件、监听、发布)
2 //定义监听类，在spring配置文件中,注册事件类和监听类
3 public class ApplicationContextListener implements ApplicationListener {
4     @Override
5     public void onApplicationEvent(ApplicationEvent event) {
6         if (event instanceof MessageSourceEvent) {
7             System.out.println("进入到监听器类---one");
8             MessageSourceEvent myEvent = (MessageSourceEvent) event;
9             myEvent.print();
10        }
11    }
12 }
```

监听二

```
1 public class ApplicationContextListenerTwo implements ApplicationListener {
2     @Override
3     public void onApplicationEvent(ApplicationEvent event) {
4         if(event instanceof MessageSourceEvent){
5             System.out.println("进入到监听器类---two");
6             MessageSourceEvent myEvent=(MessageSourceEvent)event;
7             myEvent.print();
8         }
9     }
10 }
```

3) 发布事件

```
1 //在Spring 中使用事件监听机制(事件、监听、发布)
2 //编写发布类
3 //该类实现ApplicationContextAware接口 , 得到ApplicationContext对象, 使用该对象的
  publishEvent方法发布事件
4 public class ApplicationContextListenerPublisher implements
  ApplicationContextAware {
5
6     private ApplicationContext applicationContext;
7
8     @Override
9     public void setApplicationContext(ApplicationContext applicationContext)
  throws BeansException {
10         this.applicationContext = applicationContext;
11     }
12
13     public void publishEvent(ApplicationEvent event) {
14         System.out.println("发布事件");
15         applicationContext.publishEvent(event);
16     }
17 }
```

4) 测试

配置文件

```
1 <!-- 事件发布机制 -->
2 <bean id="applicationContextListener"
  class="com.hero.observer.ApplicationContextListener"/>
3 <bean id="applicationContextListenerTwo"
  class="com.hero.observer.ApplicationContextListenerTwo"/>
4 <bean id="applicationContextListenerPublisher"
  class="com.hero.observer.ApplicationContextListenerPublisher"/>
```

```
1 //总结 : 使用bean工厂发布和使用多播器效果是一样的
2 public class ObserverTest {
3
4     public static void main(String[] args) {
5         ApplicationContext context =
6             new
7             ClassPathXmlApplicationContext("classpath*:applicationContext.xml");
8         //*****使用spring的多播器发布*****
9         //      ApplicationEventMulticaster applicationEventMulticaster =
10            (ApplicationEventMulticaster)context.getBean("applicationEventMulticaster");
11         //      applicationEventMulticaster.multicastEvent(new
12            MessageSourceEvent("测试..."));
13         //*****使用BeanFactory的publishEvent发布*****
14         ApplicationContextListenerPublisher myPublisher =
15            (ApplicationContextListenerPublisher)context.getBean("applicationContextList
16            enerPublisher");
17         myPublisher.publishEvent(new MessageSourceEvent("测试..."));
18     }
19 }
```

```
14 |  
15 }
```

多播发布

Run: ObserverTest x

信息: Loading XML bean definitions from URL [file:/D:/

进入到事件源的有参数构造器

进入到监听器类---one

进入到事件源的方法

进入到监听器类---two

进入到事件源的方法

工厂发布

Run: ObserverTest x

信息: Loading XML bean definitions from

进入到事件源的有参数构造器

发布事件

进入到监听器类---one

进入到事件源的方法

进入到监听器类---two

进入到事件源的方法

总结:

- spring的事件驱动模型使用的是观察者模式，通过ApplicationEvent抽象类和ApplicationListener接口，可以实现事件处理。
- ApplicationEventMulticaster事件广播器实现了监听器的注册，一般不需要我们实现，只需要显示的调用applicationcontext.publisherEvent方法即可
- 使用bean工厂发布和使用多播器效果是一样的